



REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology



Graduation Project

to obtain the

National Engineering Diploma

in Applied Sciences and Technology

Specialty: Instrumentation and Industrial Maintenance

Development of an IOT-based Weather Sensor System for Drone Operations

Prepared by

Mahdi MAJJEDI

Hosted by Unisphere GmbH



Defended on 25/09/2025 in front of a jury composed of:

Mr. Atef BOULILA President of the jury

Mr. Saber TLILI Reviewer

Ms. Imen JAAFAR Supervisor at INSAT

Mr. Michael ANGER Supervisor at the Company

School year: 2024/2025



REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology



Graduation Project

to obtain the

National Engineering Diploma

in Applied Sciences and Technology

Specialty: Instrumentation and Industrial Maintenance

Development of an IOT-based Weather Sensor System for Drone Operations

Prepared by

Mahdi MAJJEDI

Hosted by Unisphere GmbH



Supervisor at the Company Mr. Michael ANGER	Supervisor at INSAT Ms. Imen JAAFAR
Date:	Date:

School year: 2024/2025

Dedication

To my beloved family—my mother, father, and sister—whose love, encouragement, and unwavering belief in me have been the foundation of my journey. Your constant support gave me the strength to persevere through challenges and the motivation to keep striving for more.

To my extended family, whose kindness and support in many different ways have helped me reach this milestone and made the path smoother along the way.

To my friends, who stood by me with patience, laughter, and understanding, reminding me that even during the busiest and most difficult times, life is brighter when shared.

This work is dedicated to all of you, with gratitude for your emotional support, and with the hope that it reflects the values of learning, growth, and persistence that you have helped instill in me.

Mahdi Majjedi

Acknowledgements

*First and foremost, I would like to express my sincere gratitude to my professor, **Imen JAAFAR**, for her invaluable guidance, encouragement, and continuous support throughout this thesis project. Her expertise and thoughtful advice were essential in shaping both the technical and academic aspects of this work.*

*I would also like to thank the members of the **jury**, who dedicated their time and effort to carefully read this report and evaluate my work. Their contribution ensures that this project is assessed with fairness and rigor, and I deeply appreciate their commitment.*

*I am profoundly grateful to **Michael ANGER**, CTO of Unisphere GmbH, for giving me the opportunity to carry out my internship within the company. His trust allowed me to explore a challenging and meaningful project in a stimulating professional environment.*

*My heartfelt thanks go to **Michael SCHEER**, software developer at Unisphere, with whom I had the privilege of working closely. His supervision, availability, and constructive feedback were invaluable throughout this journey, and I benefited greatly from his expertise and mentorship.*

*Finally, I wish to express my appreciation to the entire **Unisphere team**. Their collaborative spirit, welcoming atmosphere, and unique company culture made this internship not only professionally enriching but also personally memorable.*

Abstract

This thesis presents the integration of the Vaisala Present Weather Detector 12 (PWD12) visibility sensor into an existing weather station system initially built around a Vaisala WXT536 sensor. The system is deployed on a Revolution Pi industrial controller and serves the needs of Unisphere GmbH, where high-quality, real-time meteorological data is required to support safe and efficient flight monitoring through the NOVA platform.

The work addresses both hardware and software challenges. On the hardware side, the PWD12 was connected using a dedicated Recommended Standard 485 (RS-485) interface to ensure compatibility with its baud rate and communication protocol. The wiring design prioritized simplicity and robustness, while maintaining independence between sensors to avoid interference.

On the software side, the integration followed a modular Python-based approach. Each sensor was assigned its own reader and parser modules, running concurrently under a systemd-managed service. Data acquisition, validation, and parsing pipelines were implemented to produce structured JavaScript Object Notation (JSON) payloads compatible with the existing cloud infrastructure. Particular attention was given to error handling and resilience, including checksum verification and code modularity for future extensibility.

One of the key contributions of this work is the implementation of local offline buffering through an embedded SQLite database. This ensures data persistence and reliable synchronization with the AWS AppSync GraphQL Application Programming Interface (API) even during connectivity interruptions. Retry logic and recovery workflows were designed to preserve data integrity and maintain chronological ordering of records during cloud uploads.

The results demonstrate a resilient and extensible system capable of supporting additional sensors with minimal reconfiguration. The integration of the PWD12 significantly expands the scope of collected environmental data, particularly by enabling visibility and present weather measurements. This work not only enhances the operational value of the weather station for Unisphere but also provides a modular blueprint for future sensor integrations in similar embedded cloud-connected environments.

Contents

Introduction	1
1 General Context	2
1.1 Presentation of the Hosting Company	2
1.1.1 Unisphere GmbH	2
1.1.2 Solar Impulse Project	4
1.1.3 Products and services	4
1.1.3.1 NOVA Platform	4
1.1.3.2 Operational Analysis	6
1.1.3.3 U-Space Weather Service	7
1.1.4 Company Structure and Organization	8
1.2 Project Overview	9
1.2.1 Background and Motivation	9
1.2.2 Project Context and Environment	10
1.2.3 Problem Statement and Challenges	10
1.3 Methodology	11
1.3.1 Action Plan and Approach	11
1.3.2 Gantt Chart and Timeline	11
2 Study of the Existing System	14
2.1 System Description	14
2.1.1 System Overview	14
2.1.2 Hardware Architecture	15
2.1.3 Communication Interfaces	16
2.1.4 Software Stack and Cloud Services	17
2.2 System Evaluation	18
2.2.1 System Strengths	18
2.2.1.1 Robustness and Reliability	18
2.2.1.2 Simplicity and Modularity	18
2.2.1.3 Cloud Integration	19
2.2.2 Limitations and Opportunities for Improvement	19
2.2.2.1 Lack of Visibility Measurement	19

2.2.2.2	Protocol and Communication Constraints	19
2.2.2.3	Lack of Offline Resilience	20
2.2.2.4	Data Handling and Storage Gaps	20
2.2.2.5	Scope of This Work	21
3	Study of the PWD12 Sensor	22
3.1	Sensor Context and Operational Role	23
3.1.1	Purpose of Integrating the PWD12	23
3.1.2	Role in the Weather Station	23
3.1.3	Relevance to UAS Operations and the NOVA Platform	23
3.2	Internal Architecture and Hardware Components	23
3.2.1	Optical Measurement System	23
3.2.2	RAINCAP Precipitation Sensor	24
3.2.3	Anti-Icing and Heating Mechanisms	25
3.2.4	Signal Processing Unit and Embedded Controller	25
3.2.4.1	Enclosure and Protection	25
3.3	Technical Specifications	26
3.3.1	Measurable Parameters and Units	26
3.3.2	Accuracy, Resolution, and Operating Ranges	26
3.3.3	Supported Message Types and Baud Rates	27
3.3.4	Power Requirements and Electrical Characteristics	27
3.4	Installation and Environment Requirements	28
3.4.1	Site Selection Guidelines	28
3.4.2	Mounting Orientation and Positioning	28
3.4.3	Grounding and Surge Protection	29
3.5	Electrical Wiring and Communication Interfaces	29
3.5.1	Pin-out and Wire Color Mapping	29
3.5.2	Supported Communication Interfaces	31
3.5.3	Cable Shielding and Length Constraints	31
3.6	Sensor Configuration and Command Structure	31
3.6.1	Command Mode and Access	32
3.6.2	Configuration and Setup Commands	32
3.6.3	Calibration and RAINCAP Adjustment	32
3.6.4	Diagnostic and Status Commands	33
3.7	Sensor Testing and Message Validation	33
3.7.1	Bench Setup and Mock Communication	33
3.7.2	Message Format Verification and Sensor Behavior	34
3.7.3	Raw Message Interpretation and Field Mapping	35
3.7.4	Final JSON Structure and Cloud Compatibility	36

4	Integration of the PWD12 Sensor into the Weather Station	38
4.1	System Architecture and Data Flow	38
4.1.1	Revised Hardware and Software Stack Overview	38
4.1.2	Local Communication and Parsing Workflow	40
4.1.3	Cloud Transmission Pipeline and AWS Integration	41
4.2	Codebase Structure and Development Environment	41
4.2.1	Programming Environment and Tools Used	41
4.2.2	Folder Layout and Modularization Approach	42
4.2.3	Environment Variable Management and Secrets Handling	43
4.3	Data Acquisition Pipeline	43
4.3.1	Serial Port Communication	43
4.3.2	Message Parsing and Error Handling	43
4.3.3	Schema Mapping and Output Structuring	44
4.4	Offline Buffering and Local Data Resilience	46
4.4.1	Problem Statement and Design Requirements	46
4.4.2	Evaluation of Local Storage Options	46
4.4.3	Buffer Architecture and File Management	46
4.4.4	Retry Logic and Recovery Workflow	47
4.4.5	Edge Case Handling and Failure Modes	48
4.5	Cloud Communication and Batch Upload	48
4.5.1	GraphQL Mutation Design	48
4.5.2	AppSync Pipeline Resolver for Batching	49
4.6	Deployment and Monitoring	49
4.6.1	Deployment Strategy for RevPi	49
4.6.2	Systemd Configuration and Startup Behavior	50
4.6.3	Remote Access and Diagnostics	51
4.6.4	Logging, Metrics, and Alerting	51
4.7	Documentation and Maintainability	52
4.7.1	Developer Documentation and Comments	52
4.7.2	Update Strategy and Future Expansion	52
4.7.3	Testing Guidelines and Deployment Checklist	53
4.7.4	Files to touch when adding a new sensor.	53
4.8	Results	53
4.8.1	Local Logs via Qbee	54
4.8.2	Cloud Data on AWS DynamoDB	54
4.8.3	Discussion of Results	56
4.8.4	Field Validation and Bug Fixing	56
	General Conclusion	58
	Bibliography	59

List of Figures

1.1	Unisphere’s technology-driven approach for reliable, automated flight operations from the ground to the stratosphere.[3]	3
1.2	Global flight path of Solar Impulse 2.[3]	4
1.3	Functional Architecture of the NOVA Platform	5
1.4	Input Layers and Safety Metrics Generated by the Operational Analysis Module.[3]	7
1.5	Workflow of the U-Space Weather Service.	8
1.6	Organizational Structure of Unisphere GmbH	8
1.7	The Dubai International Vertiport (DXV), adjacent to Dubai International Airport (DXB), the first commercial vertiport facility to receive technical design approval from the UAE. [7]	9
1.8	Gantt Diagram	12
2.1	System overview: functional data flow with supporting infrastructure (power, surge, remote management).	15
2.2	Data Flow Architecture	18
2.3	Proposed improvement to cloud architecture with persistent data storage	20
3.1	Vaisala Present Weather Detector PWD12	22
3.2	PWD12 Optical System Layout[8]	24
3.3	RAINCAP Block Diagram[8]	24
3.4	Recommended installation site criteria for PWD sensors (see PWD12 Manual[8], Chapter 4)	28
3.5	PWD12 cabling overview: 16-wire original mapping	30
3.6	Bench testing setup: PWD12 connected to external PSU and USB–RS485 converter	34
3.7	Example of a raw PWD12 message and the corresponding parsed JSON structure.	36
4.1	Simplified 8-pin wiring for PWD12 integration into the weather station	39
4.2	Revised hardware setup	40
4.3	Data flow within the Revolution Pi (RevPi): from sensor input to message parsing and output	41
4.4	Logigram of sensor data acquisition and validation workflow.	44
4.5	Logigram of offline buffering with SQLite.	45
4.6	Logigram of cloud transmission with retry mechanism.	47

4.7	Example of live PWD12 sensor logs captured remotely through Qbee, showing parsed data, AppSync responses, and separate buffering events.	54
4.8	PWD12 sensor data stored in AWS DynamoDB. Each row corresponds to a structured measurement record with timestamp, station identifier, and JSON payload.	55
4.9	Detailed view of a single PWD12 sensor record in DynamoDB.	55
4.10	Excerpts from live logs during the rooftop field test.	56

List of Tables

2.1	Comparison between American Standard Code for Information Interchange (ASCII) and Modbus Remote Terminal Unit (RTU) protocols	19
3.1	PWD12 Visibility Sensor Measurement Characteristics (See PWD Datasheet[9])	27
3.2	Recommended Pin-out Connections for Basic Operation	30
3.3	PWD12 Configuration Commands (see PWD12 Manual[8], Chap. 5)	32
3.4	PWD12 RAINCAP Calibration Commands (see PWD12 Manual[8], Chap. 5) .	32
3.5	PWD12 Status and Diagnostic Commands (see PWD12 Manual[8], Chap. 5) . .	33
3.6	Pin-out connections used for bench testing	34
4.1	Pin-out connections used for PWD12 integration into the weather station	39
4.2	Key development tools and technologies	42
4.3	Local storage option comparison for embedded buffering [10], [11].	46

Listings

3.1	Parsed JSON structure from raw PWD12 message	36
4.1	Project folder layout (simplified)	42
4.2	Example .env file entries	43
4.3	Parsed JSON structure from a PWD12 message	45
4.4	GraphQL schema for single-record weather data ingestion	48
4.5	Systemd unit file for PWD12 weather service	50

Acronyms

- AAM** Advanced Air Mobility. xi, 7, 9, 10, 13
- ACID** Atomicity, Consistency, Isolation, and Durability. xi, 46
- API** Application Programming Interface. iii, xi, 7, 17, 41, 43
- ASCII** American Standard Code for Information Interchange. ix, xi, 16, 17, 19, 26, 27, 32
- AWS** Amazon Web Services. iii, xi, 10, 14, 16, 17, 19, 20, 41, 45
- BVLOS** Beyond Visual Line of Sight. xi, 7
- CEO** Chief Executive Officer. xi, 8
- CI/CD** Continuous Integration and Continuous Delivery/Deployment. xi, 53
- CLI** Command Line Interface. xi, 52
- COO** Chief Operating Officer. xi, 8
- CR** Carriage Return. xi
- CRC** Cyclic Redundancy Check. xi, 16–19, 27, 31
- CTO** Chief Technology Officer. xi, 8
- DIN** Deutsches Institut für Normung. xi, 15
- DNS** Domain Name System. xi, 47
- DXB** Dubai International Airport. vii, xi, 9, 23
- DXV** Dubai International Vertiport. vii, xi, 9
- EASA** European Union Aviation Safety Agency. xi, 6, 7
- EMC** Electromagnetic Compatibility. xi, 25, 27
- EMI** Electromagnetic Interference. xi, 31
- EN61326-1** European standard for EMC requirements. xi, 25

eVTOL electric Vertical Take-Off and Landing. xi, 3, 7, 9, 10, 13

FIFO First In First Out. xi, 47

HVAC Heating, Vnetilation, and Air Conditioning. xi, 28

IoT Internet of Things. xi, 1

IP Internet Protocol. xi, 16

IP66 Ingress Protection rating 66. xi, 25, 27

JSON JavaScript Object Notation. iii, xi, 36, 40, 41, 44, 46, 47

METAR Meteorological Aerodrome Report. xi

MOR Meteorological Optical Range. xi, 24

NOTAMs Notice to Air Missions. xi, 5

NWS National Weather Service. xi, 26, 35, 36, 44

OAuth2 Open Authorization 2.0. xi

PWD12 Present Weather Detector 12. iii, vii, ix, xi, 9, 10, 15, 18, 21–31, 33, 34, 37–41, 43, 44, 53, 56, 57

REST Representational State Transfer. xi

RevPi Revolution Pi. vii, xi, 10, 11, 14–18, 38–41, 43, 54, 56

RS-232 Recommended Standard 232. xi, 31, 32

RS-485 Recommended Standard 485. iii, xi, 10, 14, 16, 17, 19, 27, 31, 34, 38–40, 43

RTA Roads and Transport Authority. xi, 9

RTU Remote Terminal Unit. ix, xi, 19

SSH Secure Shell. xi, 16, 17, 49

SYNOP Surface Synoptic Observation. xi, 35

TTL Time-to-Live. xi, 20, 48

UAM Urban Air Mobility. xi, 2, 3, 23

UAS Unmanned Aerial Systems. xi, 2, 23, 26, 38

UAV Unmanned Aerial Vehicle. xi

USB Universal Serial Bus. xi, 31

UTC Coordinated Universal Time. xi, 46

UV Ultra-Violet. xi, 25

VCS Version Control System. xi, 17

venv Virtual Environment. xi, 17

VFR Visual Flight Rules. xi, 9, 10, 13, 23, 26

WIS Weather Information Service. xi

WMO World Meteorological Organization. xi, 26, 35, 36, 44

Introduction

Weather has always been one of the most decisive factors in aviation. According to the U.S. Federal Aviation Administration, nearly 26% of flight delays are directly caused by adverse weather conditions. For traditional air traffic, this can mean rerouted flights, extended delays, or cancellations. With the rapid development of urban air mobility concepts such as air taxis and drone operations, the challenge becomes even greater: these aircraft will fly at lower altitudes, more frequently, and within densely populated environments where local weather phenomena such as fog or sudden precipitation can have an immediate impact on safety.

Unisphere GmbH is a company that develops digital solutions for flight operations and U-Space services. One of its focus areas is providing reliable and localized weather information to support decision-making for manned and unmanned aviation. This is achieved through IoT-based weather monitoring solutions, where sensors, edge devices, and cloud platforms are combined to deliver real-time insights for operators. Nevertheless, ensuring reliable data flow in field deployments remains a challenge. Weather stations must capture all relevant parameters, operate autonomously in harsh environments, and continue functioning even when network connectivity is temporarily unavailable.

In this context, the present work addresses the problem of extending and reinforcing an existing IoT weather monitoring system at Unisphere. The goal is to enhance the range of measurements, particularly by ensuring that local visibility and present weather conditions are available, and to increase the system's resilience by enabling local data buffering during connectivity disruptions. By improving both the scope and reliability of weather data, the system can better support operational decisions in aviation and emerging UAM applications.

This thesis is organized as follows. It first introduces the general context and objectives of the work, then presents the existing weather station system with its architecture, strengths, and current limitations. Next, it describes the methodology and technical steps followed for integration and implementation, before reporting on deployment and results obtained in real conditions. The document concludes with a discussion of remaining challenges and perspectives for future development.

Chapter 1

General Context

Introduction

This chapter introduces the overall context of the project. It includes a brief overview of the hosting company, its key products, and the organizational structure. This background information provides the necessary foundation to understand the environment in which the project takes place.

1.1 Presentation of the Hosting Company

1.1.1 Unisphere GmbH

Unisphere GmbH is a privately held company headquartered in Konstanz, Germany. Founded in 2017 by a team of aviation and technology experts, it employs a diverse group of engineers, data scientists, and aviation professionals, focusing on serving clients across Europe and internationally. [1], [2], [3].

Operating within the aerospace and aviation technology industry, Unisphere specializes in leveraging advanced data analytics, and software development to enhance the safety and efficiency of Urban Air Mobility (UAM) and Unmanned Aerial Systems (UAS). The company aims to contribute to the development of sustainable and automated flight technologies, addressing the growing demand for innovative air transport solutions [1], [3].

Its mission centers on enabling safe, efficient, and sustainable aviation through digital innovation, envisioning a future where automated and electric flight transform urban environments while maintaining the highest safety standards [1], [3].

Since its founding, Unisphere has gained recognition for its innovative contributions, securing multiple funding rounds and establishing strategic partnerships with key industry players. The company's experts actively participate in international aviation forums, influencing regulatory frameworks for emerging airspace technologies [1], [3], [4].

Unisphere fosters a collaborative and innovative work culture that values interdisciplinary teamwork, continuous learning, and inclusivity. The company also emphasizes sustainability

and social responsibility within its workplace practices, aligning its culture closely with its mission and vision [1], [3].

Figure 1.1 illustrates how Unisphere classifies different airspace domains based on altitude, ranging from low-level drone operations to higher-altitude electric Vertical Take-Off and Landing (eVTOL) and commercial aircraft operations. This classification reflects the company's focus on developing digital infrastructure and operational tools tailored to each layer of the airspace ecosystem, which can be summarized as follows:

- **Ground Level, Altitude < 150 m:** Focused on drone operations for various applications, including delivery and surveillance, ensuring efficient and safe operations within urban environments.
- **Altitude from 150 m to 3 km:** Dedicated to air taxis and other piloted or automated aircraft designed for UAM, facilitating quick and sustainable transportation across cities.
- **Altitude from 11 km to 20 km:** Encompasses high-altitude platforms such as solar-powered drones and airships, enabling long-endurance missions and extending connectivity and observation capabilities.

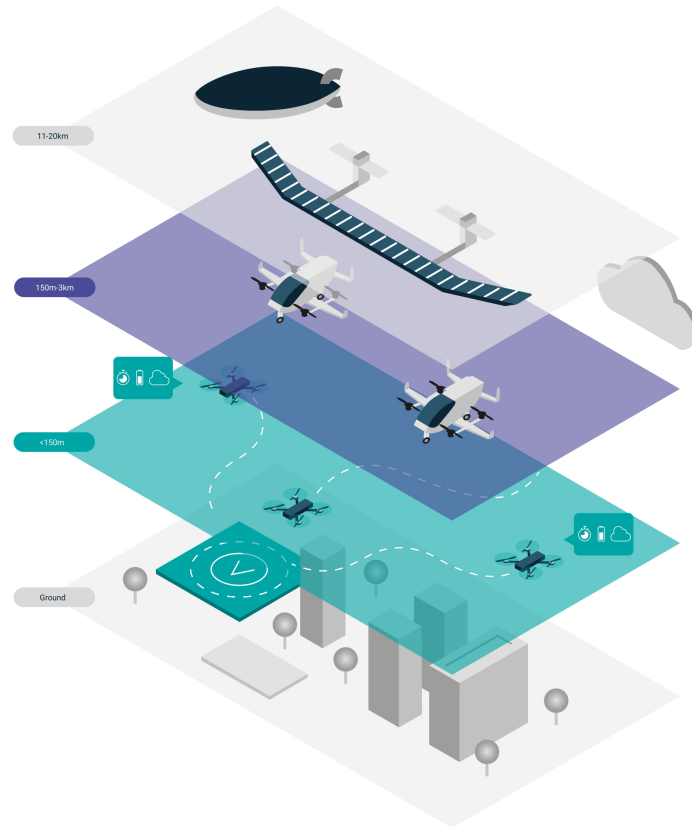


Figure 1.1: Unisphere's technology-driven approach for reliable, automated flight operations from the ground to the stratosphere.[3]

1.1.2 Solar Impulse Project

The Solar Impulse project, renowned for achieving the first solar-powered circumnavigation of the globe, played a pivotal role in the founding of Unisphere GmbH. This groundbreaking initiative, led by Bertrand Piccard and André Borschberg, showcased the potential of sustainable aviation by completing a 40,000-kilometer journey without a single drop of fuel. The aircraft, Solar Impulse 2, was equipped with 17,000 solar cells powering its electric motors and charging its batteries, enabling night flights and setting multiple aviation records.[3] [5]

The operational challenges encountered during the Solar Impulse mission, particularly the aircraft's sensitivity to weather conditions due to its lightweight, electric-propelled design, necessitated innovative approaches to flight planning and operations management. A significant portion of Unisphere's leadership and technical teams previously worked on the Solar Impulse project, where they were responsible for flight planning and operations. Their experience in simulation-based flight planning, assessing over 2,000 airports worldwide, and analyzing extensive historical weather data laid the foundation for Unisphere's development. This expertise has been instrumental in creating Unisphere's high-value services aimed at addressing real-life operational challenges in the realm of innovative air mobility.[3]

To visually represent the global scope and historical achievement of the Solar Impulse mission, figure 1.2 illustrates the full circumnavigation route undertaken by Solar Impulse 2. The map highlights key stopovers across Asia, North America, Europe, and the Middle East during its journey around the world. This mission not only demonstrated the feasibility of solar-powered flight but also laid the operational and technological foundation that would later inspire the creation of Unisphere GmbH[5].



Figure 1.2: Global flight path of Solar Impulse 2.[3]

Unisphere offers a range of digital tools and platforms designed to support the safe and efficient operation of unmanned and automated aircraft. These solutions target different aspects of flight operations, from real-time weather monitoring to advanced mission simulation and data-driven operational analysis. The main products and services are presented below.

1.1.3 Products and services

1.1.3.1 NOVA Platform

Unisphere GmbH's NOVA Operations Platform is a comprehensive solution designed to streamline and automate drone flight operations. It integrates advanced weather intelligence, central-

ized asset management, and intuitive mission planning tools to enhance safety and efficiency for professional drone airlines.[3] The NOVA platform is a modular system designed to ingest various types of data, process and analyze it in real time, and support decision-making through a user-friendly interface. Its architecture includes several functional layers: data ingestion, processing and analytics, automation and decision support, and cloud services. These components work together to provide real-time insights into weather conditions, airspace constraints, and operational risks.

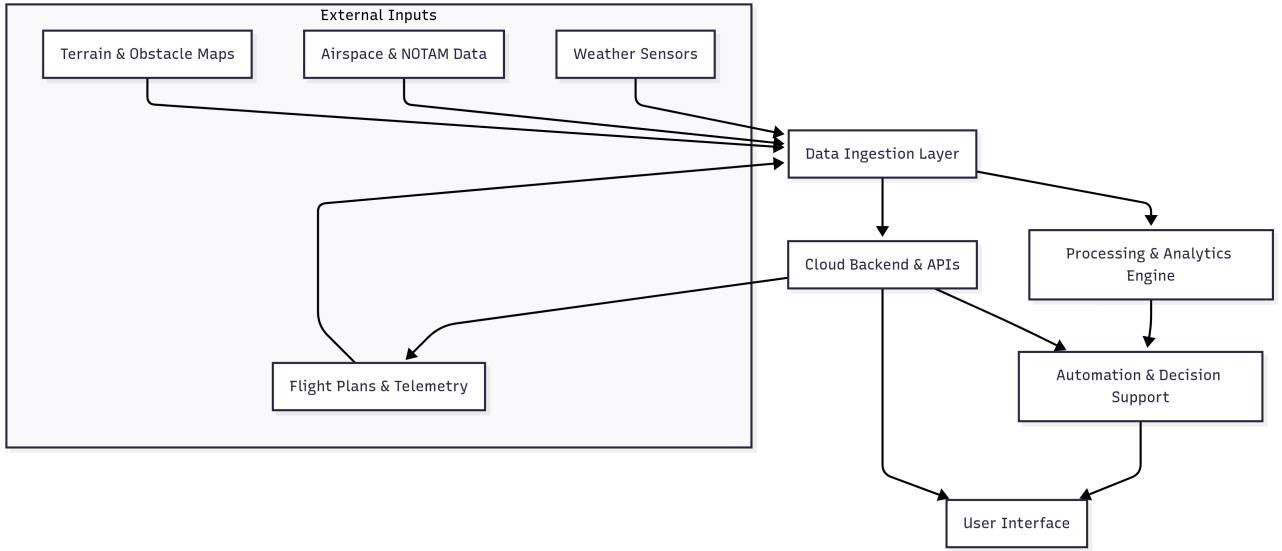


Figure 1.3: Functional Architecture of the NOVA Platform

Figure 1.3 illustrates the functional architecture of the NOVA platform, including its interaction with external data sources such as weather sensors, flight plans, Notice to Air Missions (NOTAMs), and terrain data. Each layer performs a dedicated role, ensuring secure and efficient data flow from input to user interface.

Key features of the NOVA platform include:

- **Smart Dispatching:** Consolidates all missions in one interface, enabling efficient resource management through automated weather assessments for the upcoming seven days. [3]
- **Asset Management:** Centralizes the management of various aircraft types and their operational limits, ensuring compliance across the organization. [3]
- **Team Assignment:** Provides teams with access to comprehensive weather data pertinent to their drone missions, facilitating informed decision-making. [3]
- **All-in-One Weather Tool:** Combines rain radar, weather observations, and high-resolution forecasts into a single, globally accessible platform. [3]
- **Efficient Flight Planning:** Enables quick go/no-go decisions based on automated evaluations of weather conditions in conjunction with specific mission parameters. [3]

- **Weather Intelligence:** Automatically identifies optimal flight windows with favorable weather conditions over the next 48 hours. [3]

1.1.3.2 Operational Analysis

The Operational Analysis module is part of the NOVA platform and is designed to evaluate both simulated and historical flight operations. It enables operators and engineers to assess mission feasibility, flight safety, and aircraft performance under varying environmental and regulatory constraints.[3]

This module works by combining different input data layers, such as terrain, airspace structure, aircraft characteristics, mission profiles, and meteorological conditions. These are used to simulate flight paths and compute key safety indicators, including trajectory deviation, maximum climb or sink rates, battery state-of-charge, and other aircraft limitations.[3]

Unlike the NOVA real-time simulation tools (NOW, AWARE, CAST), which support operational decision-making during or prior to a mission, the Operational Analysis module is tailored for post-flight assessment and long-term mission planning. It allows users to conduct detailed evaluations of operational performance and weather impact across timeframes ranging from a single day to an entire year.

Unisphere’s Operational Analysis supports four main types of analysis:

- **Wind Analysis:** Identifies wind characteristics by altitude, location, and time of day, providing insights into vertical flight feasibility.[3]
- **Service Availability:** Quantifies how often a given flight path or operation is feasible under weather and airspace constraints.[3]
- **Track Analysis:** Evaluates route-specific performance metrics under simulated historical weather conditions.[3]
- **Benchmark Analysis:** Compares different operational sites or conditions to identify optimal performance zones.[3]

Figure 1.4 illustrates the data flow and safety indicators generated by the Operational Analysis module. It shows how multiple input layers—including terrain, weather, aircraft data, and mission plans—are integrated to feed the simulation engine. This engine produces operational indicators that allow engineers to identify risk patterns, detect exceedance of defined thresholds, and ensure compliance with aviation standards such as the European Union Aviation Safety Agency (EASA)’s 95% usability requirement.[3]



Figure 1.4: Input Layers and Safety Metrics Generated by the Operational Analysis Module.[3]

This module plays a critical role in long-term flight optimization, infrastructure planning, and safety evaluation, especially in Advanced Air Mobility (AAM) contexts where novel aircraft and unstructured airspace introduce additional uncertainty.

1.1.3.3 U-Space Weather Service

The U-Space Weather Service is designed to meet the emerging regulatory and operational needs of drone traffic management in designated U-Space airspaces. This service seamlessly integrates real-time and forecast weather information—sourced from both local sensors and global weather models—and delivers them through a standardized API, fully compliant with EASA Regulation (EU) 2021/664 [3].

Figure 1.5 illustrates the system workflow: multiple weather service providers deliver data to Unisphere’s Weather Information Service (WIS) through interfaces such as FTP or dedicated APIs. The WIS consolidates and processes this information before exposing it via a REST API to U-Space Service Providers (USSPs) or similar traffic management systems, which in turn provide operational weather intelligence to end users. This architecture ensures that drone operators and other stakeholders can access harmonized, high-quality weather data through a single integration point.

Unisphere’s weather service enhances operational reliability in U-Space environments by automating complex meteorological calculations. It manages multiple weather variables—like wind, visibility, precipitation, and cloud ceilings—allowing operators to assess mission feasibility, flight risk, and contingency planning. As described on Unisphere’s website, this tool aims to streamline Beyond Visual Line of Sight (BVLOS) drone and eVTOL operations by delivering “best-in-class weather intelligence” through simplified integrations [3].

In addition, the service supports infrastructure-level insights by offering business-case analysis for vertiport planning. For example, it helps determine site suitability by estimating weather availability, identifying optimal approach routes, and evaluating seasonal weather patterns [3].

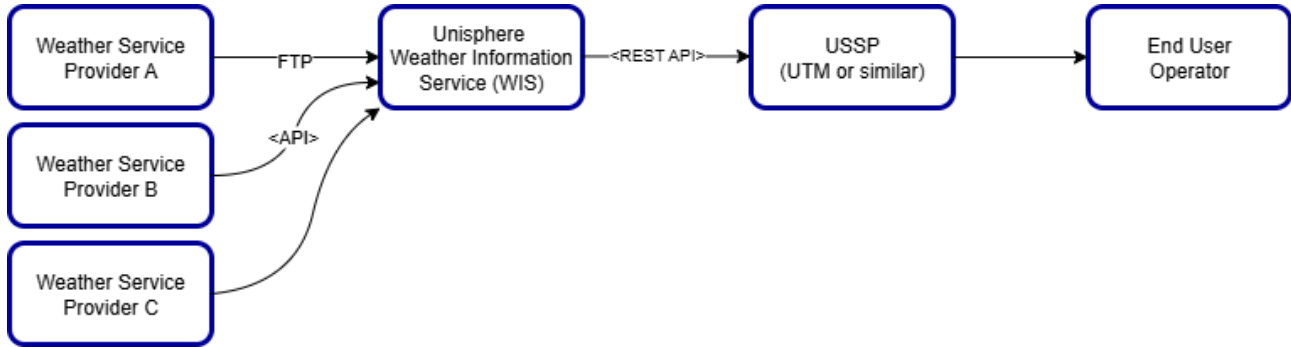


Figure 1.5: Workflow of the U-Space Weather Service.

1.1.4 Company Structure and Organization

Unisphere GmbH maintains a flat and collaborative organizational structure that supports agile development and cross-functional coordination. The company is led by an Executive Bureau composed of the Chief Executive Officer (CEO), Chief Technology Officer (CTO), and Chief Operating Officer (COO). While the CEO oversees general strategy and administration—including human resources and finance, the CTO supervises the Software Development and Engineering Teams, and the COO handles operations and sales.

The development team is divided into frontend and backend roles, while the engineering team includes domain experts such as aerospace engineers. This structure encourages continuous communication across technical and strategic departments, ensuring that product development aligns with client needs and regulatory demands.

Figure 1.6 provides an overview of Unisphere’s internal organizational structure.

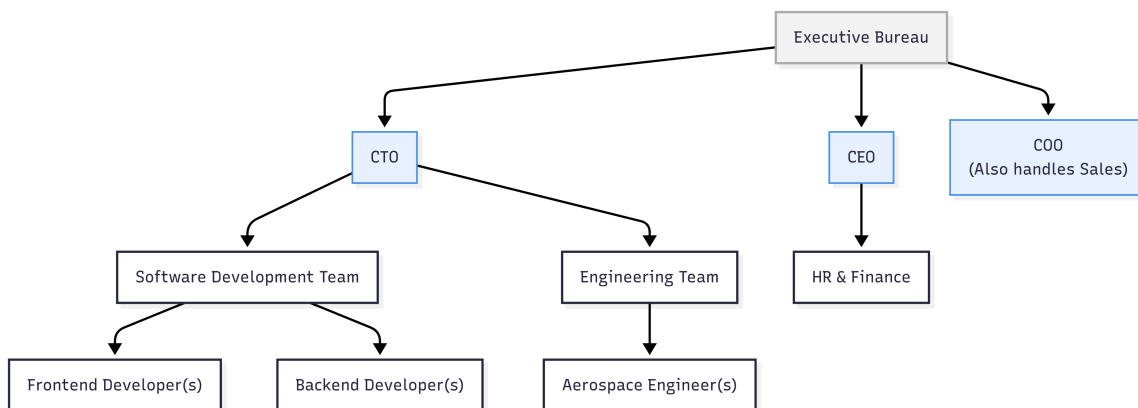


Figure 1.6: Organizational Structure of Unisphere GmbH

1.2 Project Overview

1.2.1 Background and Motivation

The integration of AAM systems, such as eVTOL aircraft, is gaining momentum globally. A key milestone in this direction is the construction of the world’s first permanent commercial vertiport near DXB, launched in November 2024. The project, led by **Skyports Infrastructure** in partnership with the **Roads and Transport Authority (RTA) of Dubai**, aims to enable air taxi services with **Joby Aviation** by early 2026. [6]

Unisphere GmbH was selected to provide the weather data infrastructure for this pioneering vertiport project through its integrated weather station system and the **NOVA platform**, which is central to Unisphere’s portfolio of drone operation management tools. **Since visibility data is a mandatory requirement** for eVTOL operations under Visual Flight Rules (VFR), the integration of a dedicated visibility sensor became essential. This provided an opportunity for Unisphere to extend its weather station capabilities, which previously focused on parameters such as wind, temperature, and precipitation, to now also include **visibility measurement** through sensors like the Vaisala PWD12.

This project fits into Unisphere’s broader strategy of continuously enhancing its simulation and operational analysis capabilities. While the vertiport use case was a primary driver, the addition of visibility data has broader applications for Unisphere’s other flight models, simulations, and operational planning tools. The goal is to provide highly accurate and localized weather data to optimize flight performance, ensure safety, and support strategic decision-making.

Figure 1.7 shows a conceptual rendering of the vertiport infrastructure being constructed in Dubai. As one of the first operational hubs for eVTOL air taxi services, this facility illustrates the real-world application context of the weather data integration carried out during the internship.

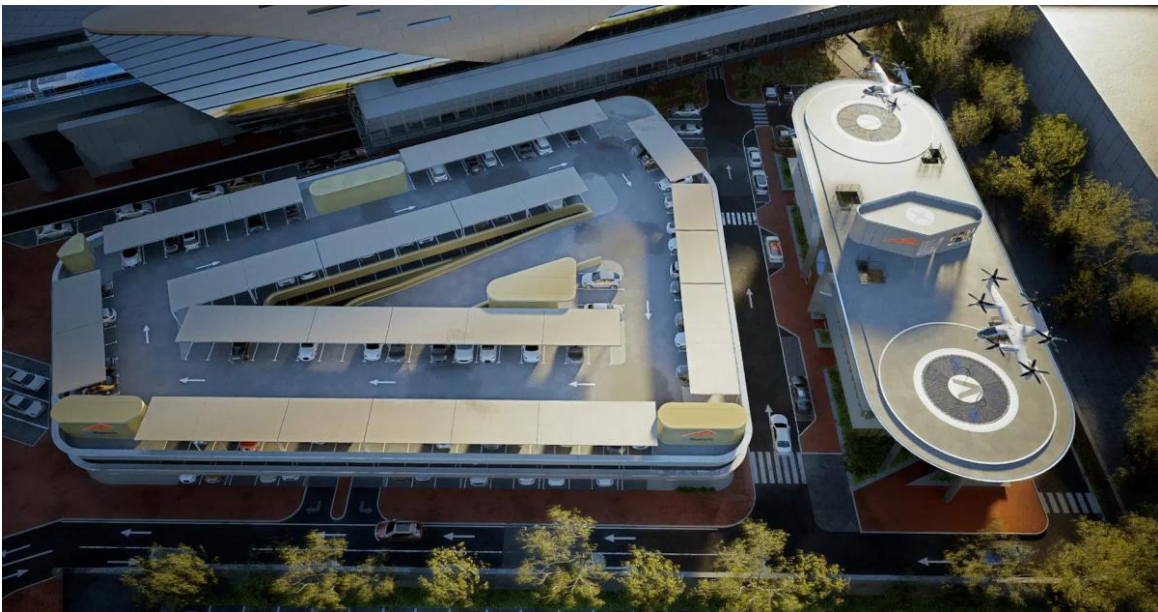


Figure 1.7: The DXV, adjacent to DXB, the first commercial vertiport facility to receive technical design approval from the UAE. [7]

1.2.2 Project Context and Environment

The integration task was conducted within the framework of an existing weather monitoring system already in use by Unisphere GmbH. This system consisted of a Vaisala WXT536 sensor, which collected weather parameters such as wind speed, temperature, humidity, pressure, and precipitation. The sensor was connected via RS-485 to a RevPi Connect+ industrial controller, where the data was parsed, validated, and transmitted to the cloud through Amazon Web Services (AWS) AppSync for use in the NOVA platform.

The system was modular and already deployed in operational settings. Its primary purpose was to provide high-resolution, localized weather data for real-time flight simulation, monitoring, and operational analysis. Due to its architecture and reliability, the RevPi-based platform was considered suitable for extending with additional functionality. However, incorporating a second sensor required careful consideration of the existing communication interfaces, data processing logic, and system scalability.

1.2.3 Problem Statement and Challenges

Despite the robustness of the existing system, a key functional limitation remained: the absence of a visibility measurement parameter, the quantitative estimation of meteorological visibility distance, expressed in meters, which is essential for eVTOL operations conducted under VFR. Under these flight rules, pilots and autonomous systems rely on sufficient visual range to navigate safely, particularly during approach and landing phases. Without accurate visibility readings, the platform could not fully support mission planning and feasibility analysis for AAM use cases such as the Dubai vertiport project.

Addressing this limitation required the addition of a dedicated visibility sensor, as the existing Vaisala WXT536 in the weather station does not measure this parameter. The chosen solution was to integrate a Vaisala PWD12, which not only measures visibility but also reports present weather conditions such as precipitation type and intensity — further enriching the dataset available for operational decision-making.

The technical integration of this second sensor posed several challenges. These included ensuring compatibility with the RevPi’s serial interfaces, managing multiple communication ports with different baud rates, and adapting the system’s data handling logic to support an additional data stream with a different message structure. Early testing efforts also faced logistical issues, such as hardware availability, power supply setup, and validating sensor outputs. Since this was the first time Unisphere integrated a Vaisala PWD12 into the system, additional effort was required to understand its communication protocol and behavior in both simulated and live conditions.

Beyond hardware and data parsing challenges, the project also introduced new expectations regarding system robustness, scalability, and cloud integration readiness. These challenges shaped the scope of the internship project and laid the foundation for the development work presented in the following chapters.

1.3 Methodology

1.3.1 Action Plan and Approach

The internship followed a structured yet flexible development approach inspired by the **Agile methodology**. Although not implemented formally using a framework such as Scrum, the work was carried out in *incremental phases*, each focusing on specific tasks such as local testing, sensor integration, system deployment, and cloud adaptation. Each phase included testing and feedback, allowing the plan to evolve based on practical findings and technical constraints.

In line with Agile principles, the process emphasized **continuous validation**, **modularity**, and **adaptability**. The work was not predefined in a rigid Gantt chart but instead progressed through short development loops, frequent test sessions, and regular updates. Informal daily exchanges with the company supervisor, combined with weekly reports sent to the academic supervisor, ensured clear communication and ongoing alignment with project objectives.

The initial steps focused on acquiring the necessary competencies, including Git usage, Linux terminal commands, and understanding the RevPi-based weather station system. This was followed by bench testing of individual components, integration of the new visibility sensor, and deployment to the actual RevPi system. Throughout the internship, a **top-down integration approach** was applied, starting from system-level understanding and gradually moving toward low-level implementation and optimization.

This combination of *iterative planning*, *real-time testing*, and *continuous feedback* provided an effective structure for carrying out a technically complex integration within a live, production-ready environment.

1.3.2 Gantt Chart and Timeline

The internship spanned a total of six months, starting on April 14th, 2025 and scheduled to end in mid-October 2025, with the final two weeks conducted remotely. The project was divided into clear, consecutive phases to ensure efficient progress and a logical development workflow. The following timeline outlines the major milestones and activities:

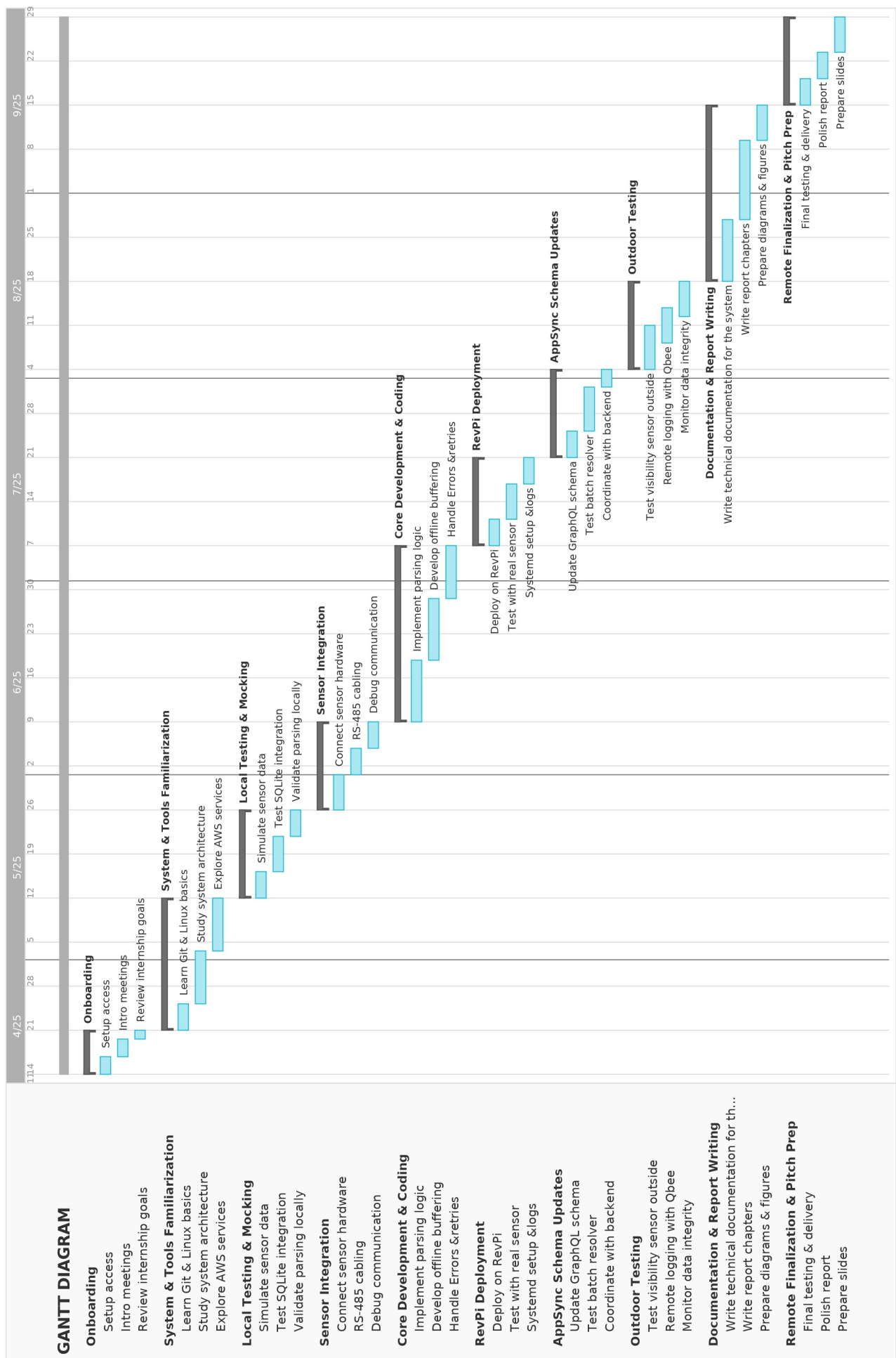


Figure 1.8: Gantt Diagram

Conclusion

This chapter introduced the operational context of Unisphere’s weather monitoring platform, the scope of the internship project, and the specific challenges addressed during the integration of a visibility sensor. The lack of visibility measurements, critical for supporting eVTOL and other AAM operations under VFR, was identified as a key gap in the existing system. To close this gap, the project set out clear objectives, defined a structured methodology, and established a realistic timeline for implementation, as outlined in the Gantt chart. The next chapter presents a detailed study of the existing system, providing the technical foundation for the integration work described later in this report.

Chapter 2

Study of the Existing System

Introduction

This chapter presents a detailed analysis of the existing weather monitoring system in place at Unisphere. The purpose is to understand its current architecture, evaluate its components, and identify its operational strengths and weaknesses. The system includes a combination of sensor hardware, embedded devices, and cloud-based services that together form a pipeline for real-time environmental data acquisition and delivery. Through this analysis, we aim to assess the efficiency, reliability, and scalability of the existing setup, and to highlight opportunities for improvement. These insights provide a foundation for the design and integration of new components in subsequent chapters.

2.1 System Description

2.1.1 System Overview

The system under study is a weather station designed to collect and transmit real-time meteorological data. It integrates a Vaisala WXT536 sensor, which measures parameters such as wind speed, temperature, humidity, pressure, and precipitation. The sensor communicates with a RevPi industrial controller over an RS-485 interface. On the RevPi, the data is parsed and formatted before being sent to the cloud via AWS AppSync. The primary goal of this system is to support Unisphere’s mission of enabling safe and optimized flight operations through the NOVA platform. By providing high-quality, real-time weather data, the system enhances situational awareness, supports trajectory planning, and contributes to safety assessments.

Figure 2.1 illustrates the overall structure of the weather station. It highlights the main data flow from the sensors to the RevPi, the cloud, and finally NOVA, while also showing the supporting infrastructure such as power supply, surge protection, and remote management tools.

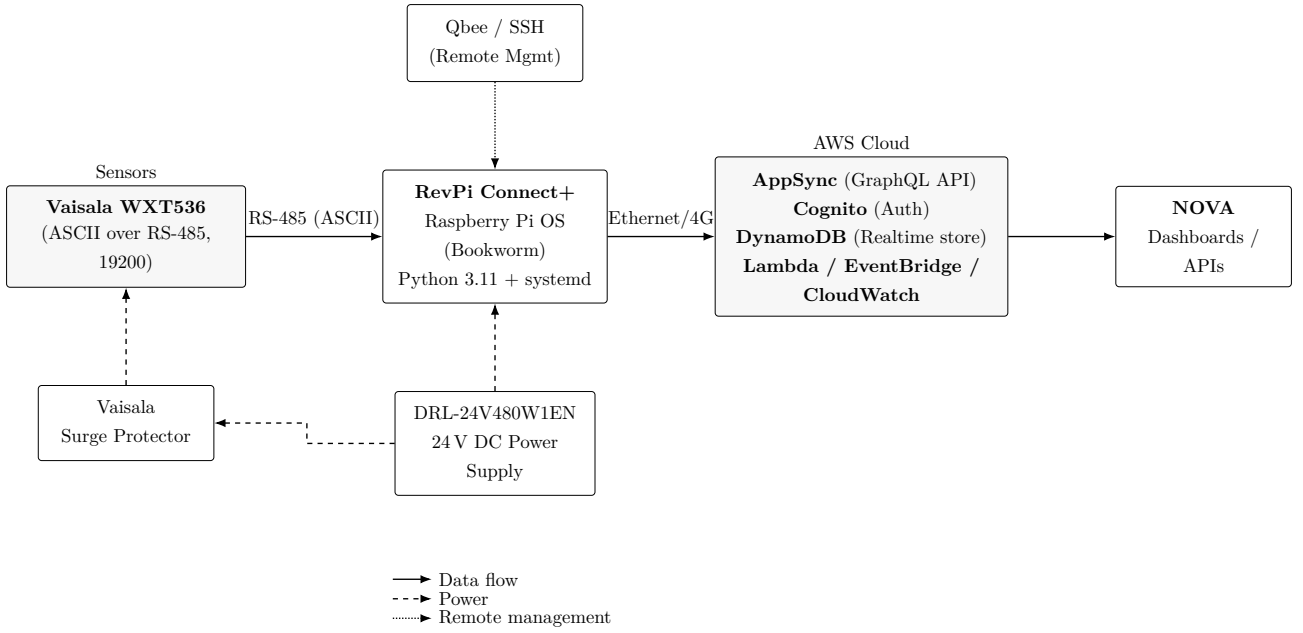


Figure 2.1: System overview: functional data flow with supporting infrastructure (power, surge, remote management).

2.1.2 Hardware Architecture

The initial deployment of the weather station system includes a compact yet robust set of components designed for industrial reliability and field deployment. The setup is composed of the Revolution Pi Connect+ as the central processing unit, connected via dedicated RS-485 interfaces to the Vaisala WXT536 and PWD12 sensors. The RevPi also connects to the 24 V DC power supply that feeds both sensors. The network connection (Ethernet/4G) provides the link to the cloud backend. This arrangement ensures that each sensor operates on an independent communication bus with the correct baud rate, reducing cross-talk and improving reliability in remote deployments.

Main Hardware Components

- **Vaisala WXT536:** Multi-parameter weather sensor measuring wind, temperature, humidity, pressure, and precipitation.
- **RevPi Connect+:** Industrial Raspberry Pi-based controller responsible for data parsing and cloud integration.
- **DRL-24V480W1EN:** Deutsches Institut für Normung (DIN)-rail industrial power supply delivering stable 24 V DC.
- **Vaisala Surge Protector:** Protects the system against lightning-induced voltage spikes.
- **Heavy-duty Mast:** A freestanding lamp stand used to mount the sensor outdoors.
- **Industrial Ethernet Switch (IDS-105FE):** Provides network connectivity between RevPi and backend services.

- **Cables and Connectors:** Including an 8-pin M12 cable for power and RS-485 communication.

Power and Communication

- **Power Input:** 230 V AC supplied to the DRL power supply.
- **Sensor Power:** 24 V DC provided to the WXT536.
- **Communication Interface:** RS-485 using an 8-pin M12 connector (differential A/B lines).
- **RevPi Serial Port:** Typically `/dev/ttyAMA5` or `/dev/ttyAMA4` depending on wiring configuration.

Installation and Protection

- **Mounting:** Pole-mounted setup suitable for testing and outdoor deployments.
- **Grounding:** Properly grounded and protected against electrical surges.
- **Enclosure:** Components placed in a sealed electrical box for weather protection.

2.1.3 Communication Interfaces

The system relies on a combination of serial and cloud-based communication interfaces to ensure seamless data flow from sensor to cloud platform.

- **Sensor-to-Controller Communication:** The Vaisala WXT536 sensor communicates with the RevPi using the **ASCII protocol** over an **RS-485** physical interface. The communication is configured at a **baud rate of 19200**. A twisted pair configuration is used for the RS-485 line, and an M12 8-pin connector is employed to connect the sensor to the electric box.
- **Data Handling on the RevPi:** Upon reception, the sensor messages are verified using a **Cyclic Redundancy Check (CRC) checksum** to ensure data integrity. Once validated, the data is parsed and formatted into a readable structure for further use and transmission.
- **Controller-to-Cloud Communication:** The parsed weather data is transmitted in real-time to the cloud using **AWS AppSync**, a GraphQL-based service that supports low-latency communication. Data is not batched, but pushed directly upon reception and validation.
- **Remote Access and Monitoring:** Remote access to the RevPi is achieved through **Secure Shell (SSH)** using its Internet Protocol (IP) address when connected to the office

Ethernet network. Additionally, **Qbee** is used as a remote monitoring and management platform, allowing secure access and deployment from external locations.

- **Surge Protection:** Although surge protection is primarily applied to the power lines, the communication setup benefits indirectly from the protected environment within the electric enclosure, reducing the risk of hardware damage.

2.1.4 Software Stack and Cloud Services

The system is built around a lightweight but robust software stack running on an industrial-grade controller, with full integration into AWS cloud services. Figure 2.2 presents a high-level view of the software and cloud architecture, illustrating the flow of data from sensor acquisition to real-time cloud transmission and backend interaction. The software and cloud components are structured as follows:

- **Programming Language:** The entire application logic is implemented in **Python**, which provides a balance between rapid development and system-level hardware communication capabilities.
- **Runtime Environment:** The RevPi Connect+ controller runs **Raspbian GNU/Linux 12 (Bookworm)**, a Debian-based operating system. The Python application is executed within a **Virtual Environment (venv)** to ensure package isolation, and is deployed as a **systemd service** to guarantee automatic startup and resilience after reboots.
- **Data Handling and Parsing:** Sensor data is received via the RS-485 interface and is subject to a **CRC checksum validation**. Once validated, the ASCII-formatted message is parsed into a structured format suitable for cloud transmission.
- **Cloud Integration:** Real-time weather data is transmitted to the cloud via **AWS AppSync**, using a GraphQL API endpoint. This allows efficient and structured interaction with backend services while enabling real-time data availability.
- **Authentication and Security:** Authentication is managed via **AWS Cognito**, and access to the AppSync endpoint is secured using an **API key**, stored in environment variables to prevent hardcoding credentials within the source code.
- **Supporting AWS Services:** In addition to AppSync and Cognito, other AWS services are integrated in the backend to support system monitoring and data processing. These include **DynamoDB** for storing incoming weather records, **CloudWatch** for runtime monitoring and logging, and **EventBridge**, which is used internally for identifying inactive sensors and automating alerting and historical weather data preprocessing.
- **Deployment and Maintenance:** For Version Control System (VCS), we use **Git**. Updates are pushed to the RevPi either via local access or remotely using **Qbee**, a device management platform that supports remote monitoring and secure SSH access.

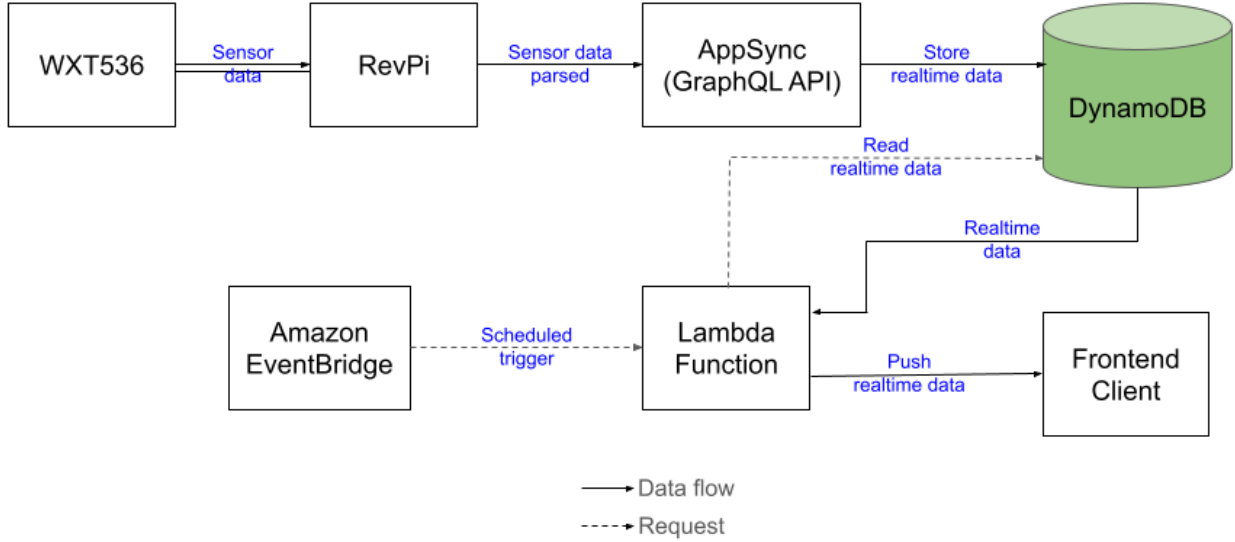


Figure 2.2: Data Flow Architecture

2.2 System Evaluation

2.2.1 System Strengths

The weather station system designed by Unisphere exhibits several strengths that contribute to its operational effectiveness, maintainability, and integration potential.

2.2.1.1 Robustness and Reliability

The existing system, prior to the integration of the new visibility sensor, was architected for operational stability and reliability in long-term deployments. The software runs as a `systemd` service on the RevPi, which ensures automatic restarts in case of unexpected shutdowns or power failures. In addition, error handling mechanisms are implemented through CRC checksum validation and structured message parsing. These features contribute to reducing the likelihood of system crashes and ensuring consistent data availability over time.

2.2.1.2 Simplicity and Modularity

The system architecture follows a modular design that separates sensor communication, data parsing, and cloud transmission into distinct functional layers. This separation allows for clear interfaces between components and simplifies both debugging and future extensions. For instance, the code responsible for parsing data from the sensor is encapsulated in independent modules, which can be reused or adapted for other sensors. The availability of an unused RS-485 port on the RevPi further simplified the physical integration of the new Vaisala PWD12 sensor, requiring minimal disruption to the existing setup. Overall, the modular structure enables scalable development without requiring a complete system overhaul.

2.2.1.3 Cloud Integration

The system benefits from seamless integration with **AWS** cloud services, particularly through the use of AppSync for real-time data streaming. This integration ensures immediate availability of weather data for backend services and frontend applications. AppSync enables efficient communication using GraphQL, which allows clients to fetch precisely the data they need, minimizing payload size and reducing response latency. The use of scalable cloud services also facilitates future extensions, such as long-term storage, alerting, and analytics.

2.2.2 Limitations and Opportunities for Improvement

While the weather station system demonstrates robustness and modularity, certain technical limitations were identified during the integration work. These limitations represent opportunities for further refinement in future system iterations.

2.2.2.1 Lack of Visibility Measurement

The existing weather station provides valuable meteorological data such as wind, precipitation, temperature, humidity, and pressure. However, it does not include visibility measurements, which are critical for aviation-related applications. Visibility data directly supports decision-making for flight safety, landing operations, and trajectory planning. The absence of this parameter reduces the system's ability to deliver a comprehensive picture of atmospheric conditions.

2.2.2.2 Protocol and Communication Constraints

The system relies on the RS-485 interface for communication with the weather sensors. While RS-485 is reliable for industrial serial communication, it requires all connected devices to operate at the same baud rate. This constraint limits the possibility of using multiple sensors with different serial configurations on the same line. Moreover, the current setup uses the ASCII protocol, which, although simple, is less efficient in terms of message size and validation. Alternative protocols like Modbus RTU may offer improvements in reliability and bandwidth usage. A comparative overview is shown in Table 2.1.

Table 2.1: Comparison between ASCII and Modbus RTU protocols

Aspect	ASCII Protocol	Modbus RTU Protocol
Message Format	Human-readable characters	Binary format
Bandwidth Efficiency	Lower (larger messages)	Higher (compact messages)
Error Detection	Typically CRC or checksum	CRC integrated
Ease of Debugging	Easier (text-based)	Harder (binary)
Message Size	Larger due to encoding	Smaller and more efficient

2.2.2.3 Lack of Offline Resilience

The original version of the weather station system was designed with the assumption of continuous internet connectivity. However, operational experience with different customer deployments revealed that connectivity disruptions can occur — for example, due to network outages or infrastructure work within the client’s network environment. In such cases, the system is unable to transmit weather data to the cloud, leading to permanent data loss during the offline period.

This limitation affects the completeness and continuity of the dataset, which are essential for use cases such as real-time monitoring, retrospective analysis, and flight simulation validation. Enhancing the system to support offline buffering and delayed transmission is therefore a critical area for future improvement.

2.2.2.4 Data Handling and Storage Gaps

The current system uses an **AWS DynamoDB** table to store incoming weather records, each of which is assigned a **Time-to-Live (TTL)** attribute of 48 hours. This design ensures that outdated entries are automatically deleted, allowing the system to maintain a lightweight and cost-efficient database while supporting real-time weather monitoring.

Although short-term retention is sufficient for most operational needs, certain use cases — such as long-term analysis, retrospective evaluations, or training of predictive models — require permanent access to historical weather data. Figure 2.3 illustrates an architectural extension that incorporates a dedicated cloud storage layer. This improvement allows the system to maintain its real-time performance while enabling historical data archiving in a scalable and structured manner.

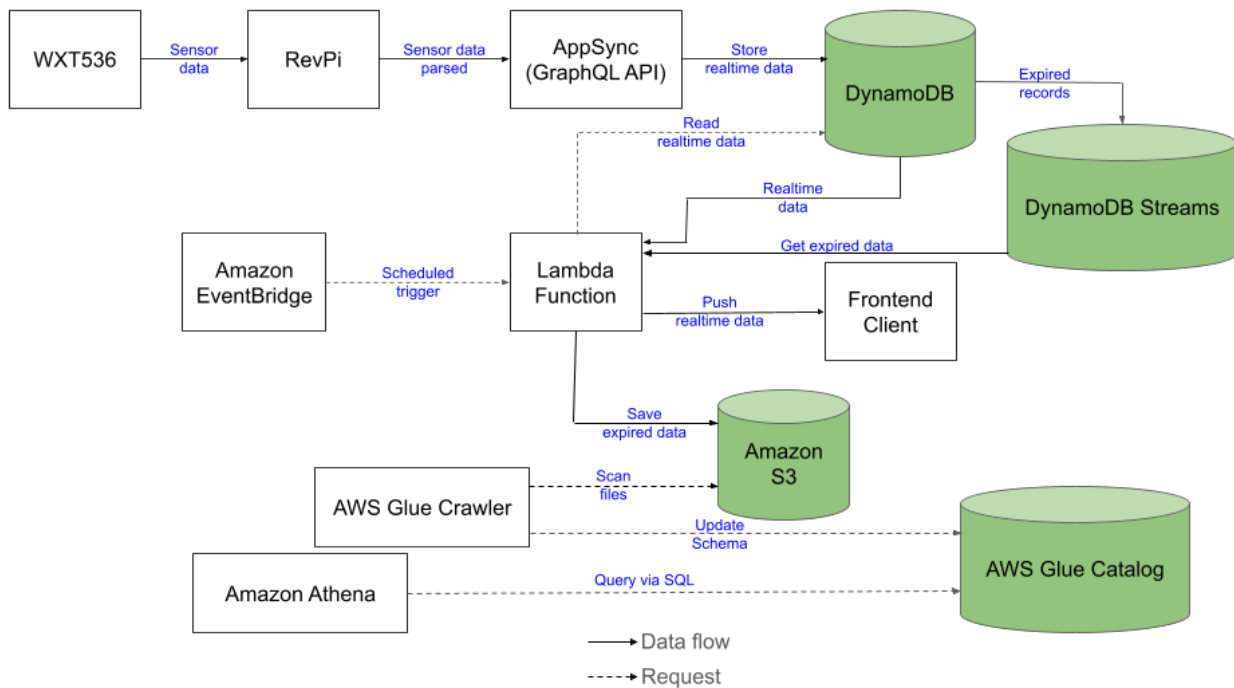


Figure 2.3: Proposed improvement to cloud architecture with persistent data storage

2.2.2.5 Scope of This Work

The analysis above highlights three main limitations of the existing weather station system: protocol and communication constraints, gaps in data storage, and lack of offline resilience. However, not all of these gaps fall within the scope of the present work.

In this thesis, two aspects are addressed directly:

- **Integration of a new visibility measurement:** The Vaisala PWD12 sensor is added to complement the WXT536 and extend the system’s measurement capabilities.
- **Enhancing offline resilience:** A local buffering mechanism based on SQLite is implemented to prevent data loss during temporary connectivity outages.

The following limitations are acknowledged but not solved within this work:

- **Protocol and communication constraints:** A theoretical improvement could be achieved by adopting Modbus RTU instead of ASCII, allowing more efficient message formats and flexible baud rate handling. This would require substantial changes in both firmware configuration and software parsing, and is therefore considered out of scope for this thesis.
- **Data storage gaps:** A potential solution is to introduce a cloud-based persistent storage layer (e.g. Amazon S3 or a time-series database) to complement DynamoDB’s short-term retention. This would enable historical analysis and model training, but lies outside the immediate objectives of this work.

This scoping ensures that the thesis remains focused on sensor integration and operational robustness, while still acknowledging other improvement paths for the system.

Conclusion

With a clear understanding of the existing weather station system, including its architecture, strengths, and limitations, the focus now shifts to the detailed study of the new visibility sensor selected for integration: the Vaisala PWD12. This chapter provided a comprehensive examination of the sensor’s internal design, operating principles, and technical specifications, as well as its relevance to the system’s operational objectives. This foundational understanding is essential to ensure effective hardware integration, data parsing, and long-term reliability within the broader weather monitoring platform.

Chapter 3

Study of the PWD12 Sensor

Introduction

This chapter focuses on the Vaisala PWD12 visibility sensor, shown in figure 3.1, a key component to be integrated into the weather monitoring system. The study covers its functionality, measurement principles, supported features, and communication capabilities. Special attention is given to the configuration process, the command interface, and how the sensor behaves in standalone testing scenarios. Understanding the PWD12's capabilities and requirements is essential before proceeding with its integration into the broader system. This chapter establishes the technical basis for ensuring reliable operation and effective data acquisition. Unless otherwise specified, all technical details in this chapter are based on the Vaisala PWD22/52 User's Guide [8].

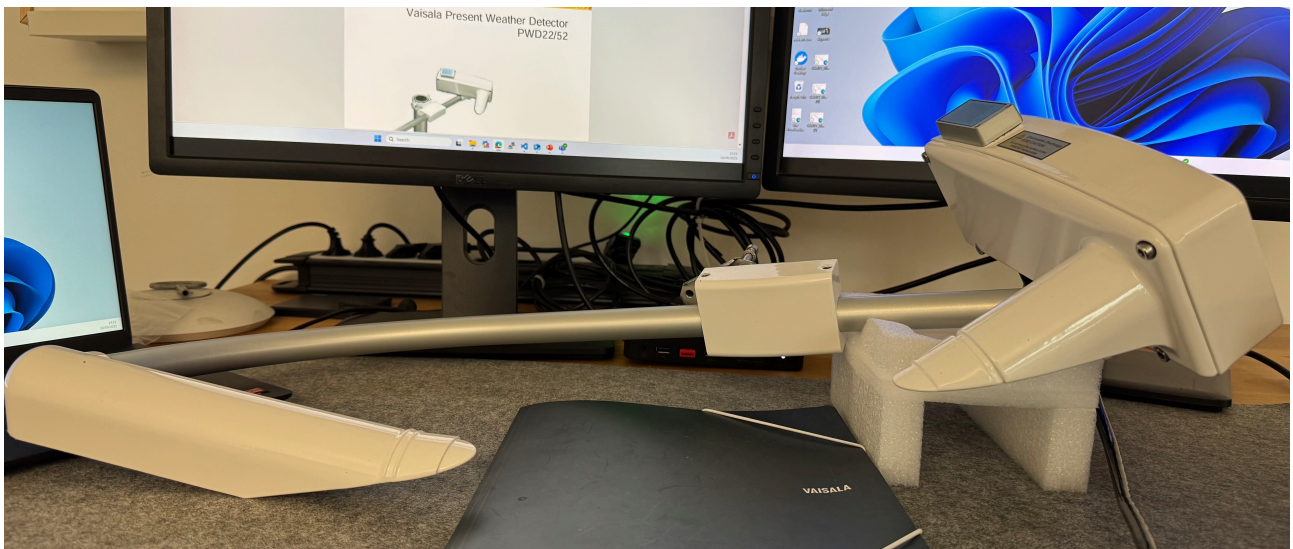


Figure 3.1: Vaisala Present Weather Detector PWD12

3.1 Sensor Context and Operational Role

3.1.1 Purpose of Integrating the PWD12

The decision to integrate the **Vaisala PWD12** visibility sensor into the weather station was largely influenced by Unisphere’s prior experience with Vaisala hardware and the reliability of their certified instruments. Vaisala’s long-standing reputation in meteorological sensing, combined with the sensor’s compliance with aviation and meteorological standards, made it a trustworthy addition to the system. Although the decision was made before the start of the internship, the integration aligned well with the company’s infrastructure and quality expectations.

3.1.2 Role in the Weather Station

The PWD12 extends the capabilities of the existing weather station by adding **visibility measurement**, a parameter that was not captured by the previously installed WXT536 sensor. This enhancement was essential for supporting operational environments such as vertiports, where visibility conditions directly impact the feasibility of air taxi operations.

3.1.3 Relevance to UAS Operations and the NOVA Platform

Visibility data plays a critical role in **UAS** operations, particularly under **VFR** and for applications like **UAM**. In such contexts, poor visibility can ground flights or require re-routing, making accurate and localized measurement a key enabler for operational decision-making.

Within the NOVA platform, visibility readings contribute to weather-aware flight planning, real-time monitoring, and post-operation analysis. The integration of the PWD12 is especially relevant for advanced vertiport deployments, such as the one near DXB, where air taxi services are planned. In these environments, visibility is not just a regulatory requirement but a critical factor in ensuring safety and operational continuity.

3.2 Internal Architecture and Hardware Components

The Vaisala PWD12 sensor is a sophisticated device designed to deliver precise visibility and present weather measurements in outdoor environments. This section explores its internal structure and the physical components that contribute to its functionality.

3.2.1 Optical Measurement System

At the core of the visibility measurement functionality is the PWD12’s **forward-scatter optical system** [8]. It consists of an infrared light source and a photodetector arranged at a 42° angle relative to each other. When light emitted by the source encounters airborne particles

like fog or rain, some of the light is scattered and captured by the receiver. The intensity of this scattered light is used to calculate the **Meteorological Optical Range (MOR)**.

This arrangement allows for a compact design with minimal maintenance requirements, as it eliminates the need for moving parts. The measurement volume is carefully defined to avoid contamination from precipitation that may pass through but not reflect light effectively.

As shown in Figure 3.2, the layout of the optical components ensures that only forward-scattered light is measured, minimizing noise from ambient light and non-relevant angles.

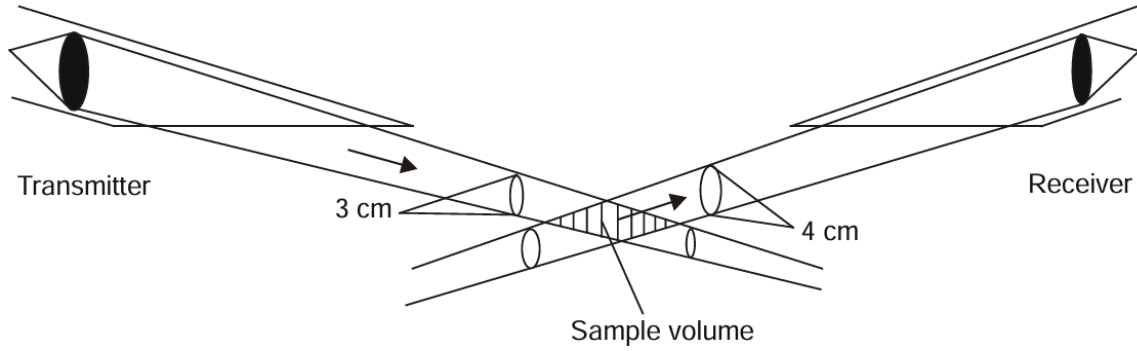


Figure 3.2: PWD12 Optical System Layout[8]

3.2.2 RAINCAP Precipitation Sensor

In parallel with visibility measurements, the PWD12 features a **RAINCAP** sensor[8] mounted on the top of the enclosure. This is a solid-state capacitive sensor that detects precipitation through mechanical vibrations caused by raindrops or snowflakes hitting its surface. The internal signal processing algorithm analyzes the impact pattern and frequency to differentiate between precipitation types like drizzle, rain, snow, and sleet.

This mechanism allows for fast and reliable precipitation detection without requiring any moving parts, enhancing durability in the field. The design and principle of operation are illustrated in Figure 3.3.

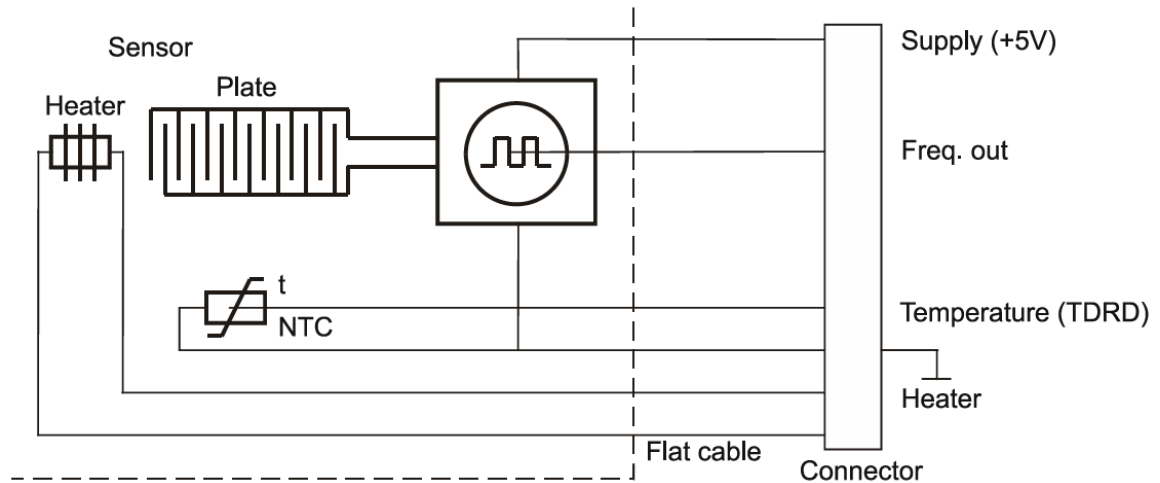


Figure 3.3: RAINCAP Block Diagram[8]

3.2.3 Anti-Icing and Heating Mechanisms

To ensure uninterrupted performance in cold and icy conditions, the PWD12 is equipped with **automatic heaters**[8] located in three zones: the optical head, the RAINCAP sensor, and the main enclosure. These heaters are activated based on environmental conditions and internal temperature readings to prevent snow or ice buildup on sensitive areas.

The heater outputs can be controlled remotely via digital I/O lines or allowed to operate autonomously. The heater configuration helps maintain consistent measurement accuracy regardless of the surrounding weather.

3.2.4 Signal Processing Unit and Embedded Controller

All sensor outputs are processed by an embedded microcontroller inside the PWD12. This unit is responsible for:

- Converting raw analog signals into digital measurements,
- Filtering and applying classification algorithms,
- Generating weather codes and output strings,
- Managing communication and diagnostic reporting.

3.2.4.1 Enclosure and Protection

The Vaisala PWD12 is housed in a rugged, **IP66-rated enclosure**[8], designed for long-term deployment in demanding outdoor environments. The enclosure combines **anodized aluminum** with **UV-resistant polycarbonate** to ensure structural integrity, weather resistance, and longevity.

Key features of the protective design include:

- **Ingress Protection:** The Ingress Protection rating 66 (IP66) rating guarantees that the enclosure is dust-tight and protected against strong water jets from any direction, making it suitable for exposed locations.
- **Electromagnetic Compatibility (EMC) Shielding:** The internal electronics are shielded to comply with **European standard for EMC requirements (EN61326-1)**, reducing susceptibility to electrical interference from nearby equipment or power lines. This is essential when the sensor is installed near radar, communication antennas, or industrial machinery.
- **Grounding and Surge Protection:** A dedicated grounding terminal allows the sensor to be connected to the site's earthing system. Additionally, the PWD12 includes internal protection components against electrostatic discharge and transient surges on communication and power lines. However, external surge arrestors are recommended for full compliance with installation best practices.

- **Environmental Robustness:** The physical form factor minimizes accumulation of snow, debris, or bird droppings on optical and sensing elements. The sloped housing design promotes self-cleaning, reducing maintenance needs.
- **Connector Interface:** The cable gland or M12 interface is sealed to prevent moisture ingress while providing mechanical strain relief for the cable.

These protection features make the PWD12 well-suited for aviation and meteorological applications where **continuous and interference-free operation** is essential.

3.3 Technical Specifications

The Vaisala PWD12 sensor is designed to provide high-accuracy visibility and present weather measurements. It outputs a variety of environmental parameters relevant for UAS operations, particularly under VFR conditions.

3.3.1 Measurable Parameters and Units

The PWD12 outputs a rich set of weather data using periodic ASCII messages[8]. The standard configuration includes the following measurements:

- **Visibility:** 1-minute and 10-minute averaged visibility values (in meters).
- **Present Weather Codes:** Instantaneous, 15-minute, and 1-hour weather conditions using both National Weather Service (NWS) and World Meteorological Organization (WMO) code standards.
- **Precipitation:** Intensity (mm/h), cumulative precipitation (mm), and snow accumulation (mm).
- **Optional Parameters:** Air temperature ($^{\circ}\text{C}$) and background luminance (cd/m^2) can also be measured when Message 7 is enabled.

3.3.2 Accuracy, Resolution, and Operating Ranges

Table 3.1 summarizes key performance parameters for the visibility and precipitation-related measurements[9].

Table 3.1: PWD12 Visibility Sensor Measurement Characteristics (See PWD Datasheet[9])

Parameter	Range	Interval	Uncertainty	Notes
Visibility (1/10 min)	1–2000 m	15 s	$\pm 10\%$ or $\pm 2.5\%$	Averaged over 1 and 10 minutes
Precipitation Intensity	0–999.99 mm/h	15 s	$\pm 2.5\%$	Resolution: 0.05 mm
Cumulative Precipitation	0–99.99 mm	15 s	$\pm 2.5\%$	Reset on restart or CLRS command
Snow Accumulation	0–999 mm	15 s	$\pm 2.5\%$	Reset on restart or CLRS command
Present Weather Code	0–99	15 s	–	Based on optical and Doppler signals

3.3.3 Supported Message Types and Baud Rates

The sensor is configured to emit **Message 2**, which contains the present weather status in standard format, and optionally **Message 7**[8] for aviation-specific use. These messages are structured in the ASCII protocol and transmitted periodically over the RS-485 interface.

Each message includes a CRC checksum for error detection, and consists of multiple comma-separated fields representing visibility, weather codes, precipitation levels, and optional extensions such as temperature and luminance.

3.3.4 Power Requirements and Electrical Characteristics

The PWD12 requires separate power inputs for its measurement electronics and its integrated heater system[8]:

- **Measurement Electronics:** 12–50 V DC
- **Heaters:** 24 V DC recommended

The sensor is designed for outdoor environments and includes built-in features for environmental resilience:

- **Operating Temperature:** -40°C to $+60^{\circ}\text{C}$
- **Ingress Protection:** IP66-rated enclosure
- **EMC Protection:** The electronics comply with EN61326-1 and EN55011 Class B emission limits.
- **Mounting:** Vertical mounting with proper optical alignment (receiver facing north in the northern hemisphere).

3.4 Installation and Environment Requirements

Proper installation is crucial to ensure the long-term reliability, accuracy, and durability of the PWD12 sensor. Vaisala provides clear guidelines to mitigate environmental interference and support optimal sensor performance.

3.4.1 Site Selection Guidelines

The sensor must be installed in an open area with unobstructed airflow and visibility. According to Vaisala’s recommendations[8]:

- A **minimum horizontal clearance of 100 m** should be maintained from large objects such as buildings, trees, or towers.
- The sensor should not be mounted near heat sources (e.g., Heating, Vnetilation, and Air Conditioning (HVAC) exhausts, rooftops) or highly reflective surfaces.
- The installation site must be accessible for maintenance and cleaning and support a vibration-free, rigid mounting base to minimize measurement artifacts.

Figure 3.4 from the official manual illustrates the optimal site selection constraints.

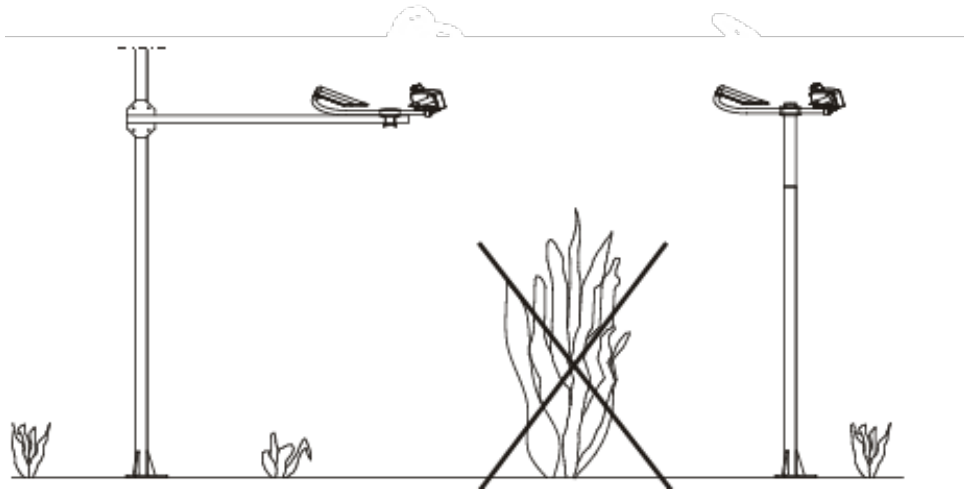


Figure 3.4: Recommended installation site criteria for PWD sensors (see PWD12 Manual[8], Chapter 4)

3.4.2 Mounting Orientation and Positioning

The PWD12 is typically mounted on a vertical pole or mast using Vaisala’s dedicated mounting kit[8] or a compatible adapter. The optical head must remain perfectly horizontal during operation to avoid distortion in visibility measurement. The mounting setup should ensure:

- A fixed, stable structure such as a mast or rail.

- Horizontal alignment of the sensor body and unobstructed view of the measurement volume.
- To avoid sunlight or artificial light interference, the optics should not face strong light sources.
- In the northern hemisphere, the **receiver should be oriented toward the north**, while in the southern hemisphere it should face south.

3.4.3 Grounding and Surge Protection

Electrical protection is essential, especially in outdoor installations exposed to lightning or power surges. Vaisala recommends the following protection measures[8]:

- The PWD12 sensor chassis must be **properly grounded** to earth using a dedicated grounding wire.
- A Vaisala **lightning protection device** should be installed in-line between the power source and the sensor, especially in high-risk areas.
- Shielded cables and grounded cable armor are recommended to further suppress electromagnetic interference.

Failing to implement grounding and surge protection can compromise both safety and long-term sensor performance.

3.5 Electrical Wiring and Communication Interfaces

The PWD12 sensor requires careful attention during electrical integration to ensure reliable communication and power delivery. This includes proper pin-out matching, interface selection, cable shielding, and adherence to electrical specifications. For complete wiring specifications, see the Vaisala PWD12 User Manual, Section 4.2[8].

3.5.1 Pin-out and Wire Color Mapping

The sensor is delivered with a fixed 16-wire cable, which provides lines for power, serial communication, relay outputs, and diagnostics. Table 3.2 shows the color mapping of the 16 wires in the PWD12 cable, the function of each wire, and the assigned pin:

Table 3.2: Recommended Pin-out Connections for Basic Operation

Wire Color	Function	Pin
Red	Power +24 VDC	1
Black	Power GND	2
White	RS-485 B (-)	3
Brown	RS-485 A (+)	4
Green	RS-232 Tx	5
Yellow	RS-232 Rx	6
Grey	RS-232 GND	7
Blue	Analog Output	8
White/Green	Heating Power +	9
Brown/Green	Heating Power +	10
White/Yellow	Heating Power -	11
Brown/Yellow	Heating Power -	12
Grey/Pink	Relay Control 1	13
Red/Blue	Relay Control 2	14
Violet	Relay Control 3 / Ext Vb	15
Pink	Ext Vb	16

Figure 3.5 shows the full 16-pin connector layout that should be used in this installation.

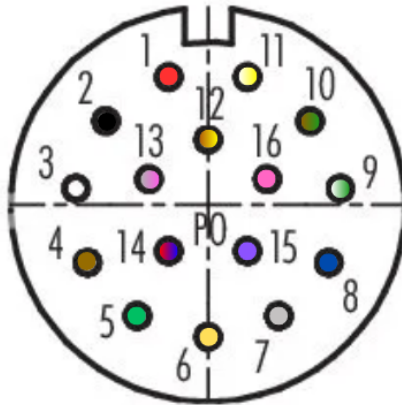


Figure 3.5: PWD12 cabling overview: 16-wire original mapping

3.5.2 Supported Communication Interfaces

The PWD12 supports two digital communication interfaces:

- **RS-485 (default):** Full-duplex, differential communication. Recommended for long distances and electrically noisy environments. Baud rate: **9600, 7 data bits, even parity, 1 stop bit**.
- **Recommended Standard 232 (RS-232) (optional):** Used primarily for manual configuration and diagnostics. Requires a Universal Serial Bus (USB)–RS-232 converter and a terminal emulator on the host device.

RS-485 was selected for this integration due to its superior resistance to electromagnetic interference, support for longer cable runs, and compatibility with existing RevPi serial interfaces. RS-232 was used only during initial bench testing and manual configuration.

3.5.3 Cable Shielding and Length Constraints

To ensure stable signal quality and Electromagnetic Interference (EMI) protection:

- Use shielded twisted pair cables for RS-485 signals.
- The cable shield (bare wire) should be grounded at the panel side only.
- Keep the communication cable length under **1,200 meters** for RS-485, as per standard limitations.
- Ensure separation between power lines and communication lines to reduce noise coupling.

These precautions improve data integrity and prevent CRC parsing errors in electrically noisy or outdoor environments[8].

3.6 Sensor Configuration and Command Structure

The Vaisala PWD12 sensor offers flexible configuration through a serial interface, primarily using RS-232 for direct command access. Most configuration and diagnostic operations are executed in *command mode*. The required serial parameters were outlined in Section 3.5.2, and this section focuses on the command protocol and available setup options.

During integration, the default settings were largely sufficient. Only minor adjustments were made to the output format (Message 2/7 selection), output interval, and filtering level using documented commands. All commands follow the syntax and descriptions provided in the Vaisala PWD12 User Manual (e.g., Section 7.3)[8].

3.6.1 Command Mode and Access

To access command mode via RS-232, the following command must be sent:

OPEN *

Once command mode is active, the sensor accepts ASCII-based instructions. All commands must be terminated with a carriage return (CR, 0x0D). The sensor replies with either confirmation messages or returned data strings depending on the command type.

3.6.2 Configuration and Setup Commands

Table 3.3 summarizes common setup commands used to control sensor behavior and output characteristics.

Table 3.3: PWD12 Configuration Commands (see PWD12 Manual[8], Chap. 5)

Command	Description
WSET	View or modify weather message type and code format (NWS/WMO).
SSET	Configure output interval, filtering, and enabled parameters (e.g., temperature, luminance).
CLRS	Clear accumulated rain and snow measurements. Used at installation or reset.
WPAR	Display current weather-related parameters such as visibility, precipitation thresholds.
MSG	Select which output messages to emit (e.g., Message 2 or Message 7).

3.6.3 Calibration and RAINCAP Adjustment

The RAINCAP sensor can be recalibrated if necessary. Table 3.4 lists relevant commands.

Table 3.4: PWD12 RAINCAP Calibration Commands (see PWD12 Manual[8], Chap. 5)

Command	Description
DRY ON	Set reference level for no precipitation. Run when the sensor is dry.
WET ON	Set reference level for active precipitation. Run during rain.
DRY / WET	Read current dry/wet calibration status and thresholds.

3.6.4 Diagnostic and Status Commands

The sensor provides various commands to monitor internal status and ensure healthy operation:

Table 3.5: PWD12 Status and Diagnostic Commands (see PWD12 Manual[8], Chap. 5)

Command	Description
STAT	Returns internal status and error flags.
TEMP	Displays current internal and ambient temperatures.
IDN?	Query the sensor’s model ID, firmware version, and serial number.

These diagnostic tools support field validation, health monitoring, and automated sensor management. Most of the commands listed above are covered in Section 7.3 and Appendix A of the PWD12 User Manual.

Wherever possible, configuration changes were minimized to preserve factory-calibrated accuracy. The default Message 2 format and 15-second output interval proved sufficient for integration with the RevPi data pipeline and downstream GraphQL schema.

3.7 Sensor Testing and Message Validation

To ensure proper functionality and compatibility of the PWD12 sensor before integration into the operational weather station, it is essential to carry out standalone testing. This includes establishing a communication link, analyzing output message formats, and validating that all expected parameters are correctly reported and parsable.

3.7.1 Bench Setup and Mock Communication

For standalone testing outside of the field deployment, the PWD12 was connected to a controlled bench setup using a regulated laboratory power supply and a USB–RS485 converter attached to a development laptop. This allowed direct communication with the sensor before integrating it into the RevPi system.

The external power supply delivered 24 V DC, sufficient to power the measurement electronics. Only the essential lines from the PWD12 16-wire cable were connected: power (red/black) and the differential pair for RS-485 communication (brown/white). The heaters and additional auxiliary lines were not used in this configuration.

Table 3.6 summarizes the connections made during the bench setup.

Table 3.6: Pin-out connections used for bench testing

PWD12 Wire Color	Function
Red	Power +24 VDC
Black	Power GND
Brown	RS-485 A (+)
White	RS-485 B (–)

Figure 3.6 illustrates the simplified bench setup, showing the PWD12 sensor connected to the external power supply and to a laptop via USB–RS485 converter.

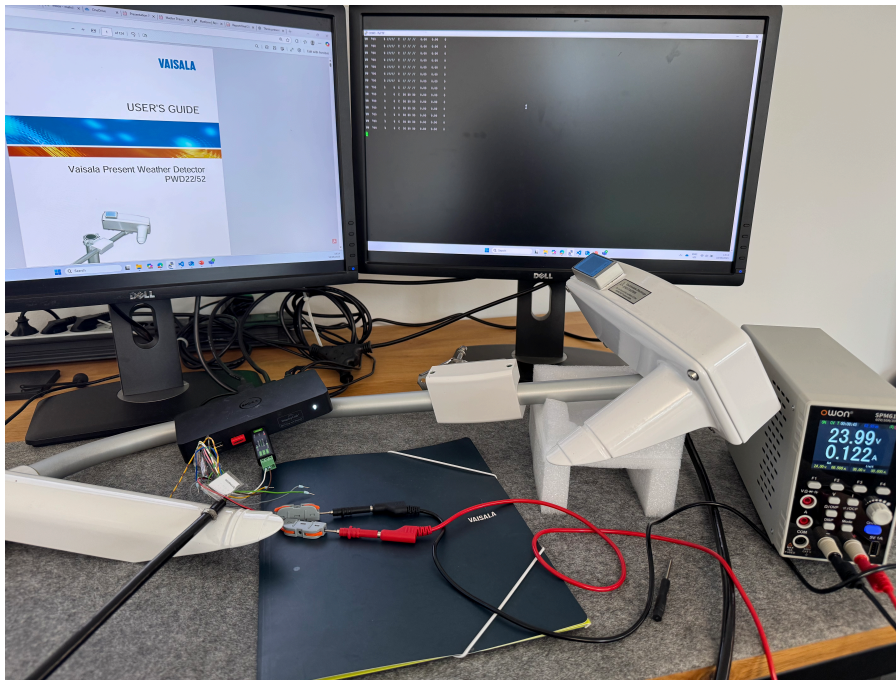


Figure 3.6: Bench testing setup: PWD12 connected to external PSU and USB–RS485 converter

Before switching to VS Code, data was monitored using a terminal emulator (**Putty**), with serial settings matching the PWD12 specification: 9600 baud, 7 data bits, even parity, and 1 stop bit.

3.7.2 Message Format Verification and Sensor Behavior

Once powered and initialized, the PWD12 begins periodically transmitting data via the RS-485 interface. By default, this output is in Message 2 format, which includes visibility readings, present weather codes, and precipitation data.

A representative message output is shown below:

```
PW 100 1839 1505 C 61 61 61 0.33 12.16 0
```

Each field in the message corresponds to a specific parameter:

- PW — Sensor prefix
- 1 — Sensor unit ID
- 00 — Status code
- 1839 — 1-minute average visibility (m)
- 1505 — 10-minute average visibility (m)
- C — Instant present weather (NWS code)
- 61, 61, 61 — Instant, 15-min, and 1-hour WMO Surface Synoptic Observation (SYNOP) weather codes
- 0.33 — Precipitation intensity (mm/h)
- 12.16 — Cumulative water sum (mm)
- 0 — Cumulative snow sum (mm)

During testing, this message was confirmed to:

- Be emitted at the expected interval (typically every 15 seconds)
- Contain no malformed syntax or communication errors
- Include all expected measurement fields

3.7.3 Raw Message Interpretation and Field Mapping

To ensure compatibility with downstream processing and the cloud API, a dedicated Python parser was implemented to interpret raw messages. The parser handled:

- Serial readout and newline stripping
- Splitting by whitespace
- Assigning each field to a semantically meaningful key
- Type validation (e.g., casting to float or integer)

- Out-of-range values

This structured pipeline guarantees that data entering the cloud is both semantically valid and consistent across all weather stations.

Conclusion

This chapter provided a comprehensive study of the Vaisala PWD12 sensor, outlining its measurement principles, internal components, technical specifications, and operational requirements. Particular emphasis was placed on understanding the optical forward-scatter method for visibility estimation, the RAINCAP precipitation detection system, and the role of integrated heating and protective mechanisms in ensuring continuous operation under harsh weather conditions.

Through standalone bench testing, the sensor's communication interface and output format were successfully verified. Raw ASCII messages were captured, parsed, and transformed into structured JSON objects, demonstrating compatibility with downstream data handling pipelines. The tests confirmed that the sensor reliably outputs visibility, precipitation, and present weather codes at the expected intervals, with no communication errors or malformed data.

By establishing both the theoretical background and practical validation of the sensor's output, this study laid the groundwork for its integration into the operational weather station. The findings ensure confidence that the PWD12 can extend the system's capabilities with accurate visibility data, thereby supporting weather-aware flight monitoring and vertiport operations. The next chapter builds on these results by addressing the integration process, covering the hardware wiring, software architecture, and cloud connectivity required to deploy the sensor in the field.

Chapter 4

Integration of the PWD12 Sensor into the Weather Station

Introduction

This chapter details the integration process of the Vaisala PWD12 visibility sensor into the existing weather station system, which already featured a Vaisala WXT536 for meteorological data collection. The integration covered both hardware and software aspects, requiring careful adaptation of the system architecture to support a second sensor with different communication and data characteristics. Emphasis is placed on maintaining modularity, ensuring system reliability, and enabling cloud connectivity through AWS services. The integration not only enhances the scope of environmental data collected, particularly in terms of visibility and present weather conditions, but also strengthens the platform's value for Unisphere's flight monitoring and UAS operations. Each section of this chapter walks through the technical steps, challenges, and solutions involved in enabling seamless communication, data acquisition, offline buffering, and cloud synchronization for the new sensor.

4.1 System Architecture and Data Flow

4.1.1 Revised Hardware and Software Stack Overview

With the integration of the Vaisala PWD12 sensor, the weather station now incorporates two separate sensing units: the existing WXT536 for core weather measurements and the PWD12 for visibility and present weather detection. Both sensors are physically connected to the same RevPi device using dedicated RS-485 serial interfaces. The WXT536 operates at 19200 bps while the PWD12 uses 9600 bps, which necessitates independent communication ports for reliable operation.

In terms of cabling, the PWD12 is connected to the RevPi through its 8-pin M12 connector. Only the essential lines are used: the power supply (24 V DC) and the RS-485 differential pair for serial communication. Optional connections such as heater power or analog outputs were

not required in this deployment and were therefore left unconnected.

Table 4.1 summarizes the wiring used for the integration, while Figure 4.1 illustrates the simplified pinout layout of the 8-pin connector as implemented in the weather station.

Table 4.1: Pin-out connections used for PWD12 integration into the weather station

M12 Pin	Wire Color (Cable)	Function	PWD12 Internal Wire
2	Brown	+24 V DC Power	Red
5	Grey	RS-485 A (+)	Brown
7	Blue	RS-485 B (-)	White
8	Red	Power GND	Black

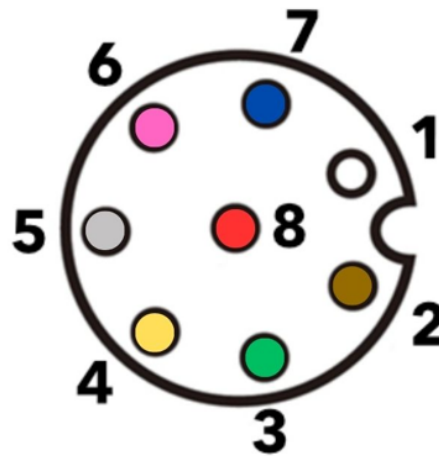


Figure 4.1: Simplified 8-pin wiring for PWD12 integration into the weather station

On the software side, the architecture retains its modular design. Each sensor is handled by its own Python module, responsible for reading, parsing, and structuring messages. These modules run in parallel threads within a single systemd-managed process, ensuring fault isolation and efficient concurrency.

Figure 4.2 presents the Updated hardware architecture of the weather station after PWD12 integration, showing the RevPi Connect+, dual RS-485 connections (WXT536 and PWD12), and supporting power supply.

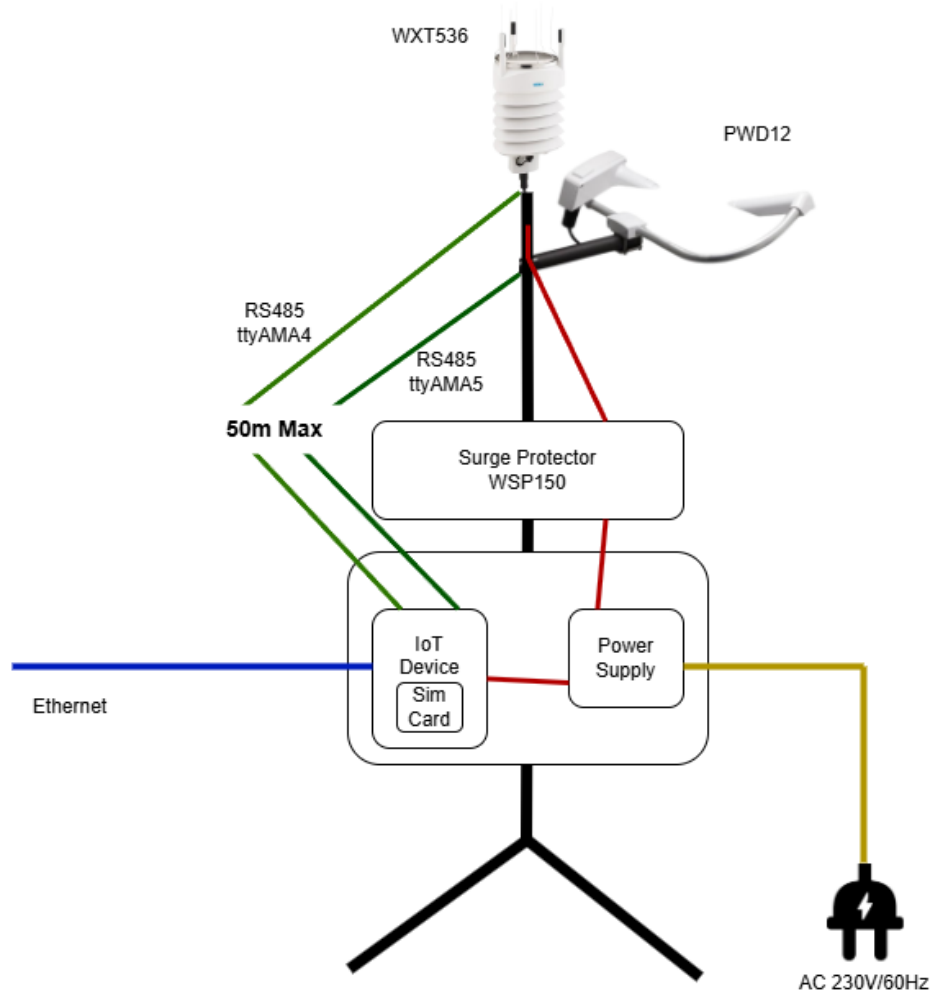


Figure 4.2: Revised hardware setup

4.1.2 Local Communication and Parsing Workflow

At the local level, the RevPi handles real-time communication with both the PWD12 and WXT536 sensors via two separate RS-485 ports. Each sensor's data stream is processed independently through dedicated reader and parser modules, which are initialized in separate threads within a single Python process managed by systemd. This design ensures that failures in one sensor's parsing logic do not interfere with the operation of the other.

The reader modules continuously monitor their respective serial interfaces, decoding messages based on predefined formats. After basic validation, the data is passed to parser modules that extract relevant fields such as visibility, precipitation, wind speed, or atmospheric pressure. Parsed messages are then structured into unified JSON-like dictionaries and passed downstream for cloud transmission or local buffering.

This modular threading architecture allows for concurrent execution while keeping the code-base maintainable and sensor logic loosely coupled. Figure 4.3 summarizes the local data flow within the RevPi: independent serial reading threads parse WXT536 and PWD12 messages into structured JSON dictionaries for either cloud upload or local buffering.

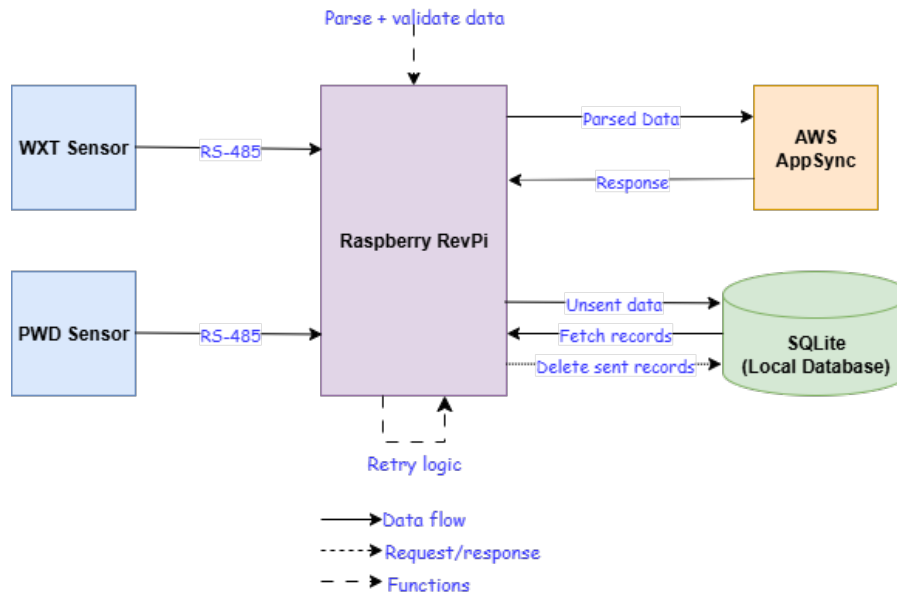


Figure 4.3: Data flow within the RevPi: from sensor input to message parsing and output

4.1.3 Cloud Transmission Pipeline and AWS Integration

Once data is parsed and structured locally, it is transmitted to the cloud via a GraphQL API exposed by AWS appsync. Both the PWD12 and WXT536 sensors share the same endpoint, with each message tagged by a sensor-specific identifier to allow differentiation on the backend. This unified interface simplifies the cloud integration and reduces the need for multiple endpoints or API configurations.

The cloud pipeline mirrors the architecture described in Chapter 2, specifically Figure 2.2, which illustrates the key services involved: appsync acts as the entry point, forwarding validated messages to a dynamodb table for temporary storage and to other backend services such as Lambda for further processing.

This design enables near real-time access to weather and visibility data while maintaining flexibility for downstream analytics or alerting procedures. The structure also supports asynchronous operations like offline buffering and retry mechanisms, which are detailed later in Section 4.4.

4.2 Codebase Structure and Development Environment

4.2.1 Programming Environment and Tools Used

The weather station system is developed in **Python 3.11** and runs on **Raspberry Pi OS (Bookworm)** installed on the RevPi Connect+. This environment offers a stable and flexible base, with full support for modern Python features and libraries.

The system relies on a minimal set of external dependencies, declared in a dedicated `requirements.txt` file. These include modules for serial communication, environment variable loading, GraphQL API interaction, and JSON processing. In addition, runtime and de-

ployment are supported by standard Linux services and external tools for version control and remote management.

Table 4.2 summarizes the key tools and technologies used, organized by category to reflect their different roles in the system.

Table 4.2: Key development tools and technologies

Category	Tool	Function
Python Libraries	<code>pyserial</code>	Serial port communication
	<code>requests</code>	HTTP-based GraphQL communication
	<code>python-dotenv</code>	Environment variable loading
System & Runtime Tools	<code>systemd</code>	Persistent background service management
Deployment & Remote Management	<code>git</code>	Version control and deployment
	Qbee	Remote access, configuration, and logging

4.2.2 Folder Layout and Modularization Approach

The codebase is organized in a modular structure to improve clarity, reusability, and separation of concerns. Each sensor has its own folder containing the core scripts for reading and parsing serial messages. Shared utilities such as offline buffering or main orchestration logic reside in the project root directory.

Listing 4.1: Project folder layout (simplified)

```
weather_station/
|- docs
|  |- images
|  |- sensor_installation.md
|  |- sensor_specifications.md
|  \- system_architecture.md
|- pwd12/
|  |- error_handler.py
|  |- parser.py
|  \- reader.py
|- wxt536/
|  |- checksum_handler.py
|  |- parser.py
|  \- reader.py
|- buffer_manager.py
|- main.py
|- data_structure.md
|- env_var.env
|- README.md
|- requirements.txt
\-.gitignore
```

This separation allows sensor-specific logic to be maintained independently, with minimal coupling. Shared logic such as error handling or buffering is centralized to prevent duplication.

4.2.3 Environment Variable Management and Secrets Handling

To avoid hardcoding sensitive information such as API keys and serial port paths, the system loads its configuration from a `.env` file using the `python-dotenv` package. This approach enables secure and flexible deployments, especially when using version control.

Listing 4.2: Example `.env` file entries

```
APPSYNC_ENDPOINT=https://your-api.apps-sync-api.amazonaws.com/graphql
APPSYNC_API_KEY=your-secret-key
PWD_ID=IPC2100_PWD12
WXT_ID=IPC2100_WXT536
WXT_PORT=/dev/ttyAMA4
PWD_PORT=/dev/ttyAMA5
```

These variables are injected at runtime and accessed by the code using standard environment access functions, providing a clean interface between configuration and logic.

4.3 Data Acquisition Pipeline

This section describes how sensor data from the WXT536 and PWD12 devices is acquired, parsed, and structured locally before being forwarded to cloud infrastructure or buffered during offline periods.

4.3.1 Serial Port Communication

The two sensors communicate independently over dedicated RS-485 interfaces, each connected to its own serial port on the RevPi. The WXT536 operates at 19200 baud with 8 data bits and no parity, while the PWD12 uses 9600 baud, 7 data bits, even parity, and one stop bit. Serial port parameters are explicitly set using Python's `pyserial` library at runtime.

The PWD12 line reader opens the port using environment variables (via `dotenv`) and waits for newline-terminated messages in blocking mode. The WXT536 reader uses a higher-level class, `WXT530ASCII`, to issue ASCII-based polling requests and receive either composite or multi-line data messages. Each read loop runs in its own thread, continuously reading and timestamping data from the corresponding port.

4.3.2 Message Parsing and Error Handling

Once data is read from the serial port, each message is validated and parsed.

- **WXT536:** Each multi-line response is parsed individually using a checksum validation function before converting key-value pairs into a structured dictionary. Custom mappings

translate sensor codes (e.g., `Sm=`, `Ta=`) into descriptive field names like `wind_speed_average` and `air_temperature`.

- **PWD12:** Each line is first validated for structure and length using a dedicated validator. The parser decodes the byte stream, filters control characters, and maps numeric fields to visibility, precipitation, and present weather codes. External lookup dictionaries are used for translating NWS character codes and WMO numeric codes into human-readable weather descriptions.

Errors such as malformed lines, invalid values, or checksum mismatches are logged and do not crash the main pipeline. For the WXT536, retry logic is included to tolerate transient checksum failures (up to 3 attempts).

Figure 4.4 illustrates this parsing and validation workflow step by step.

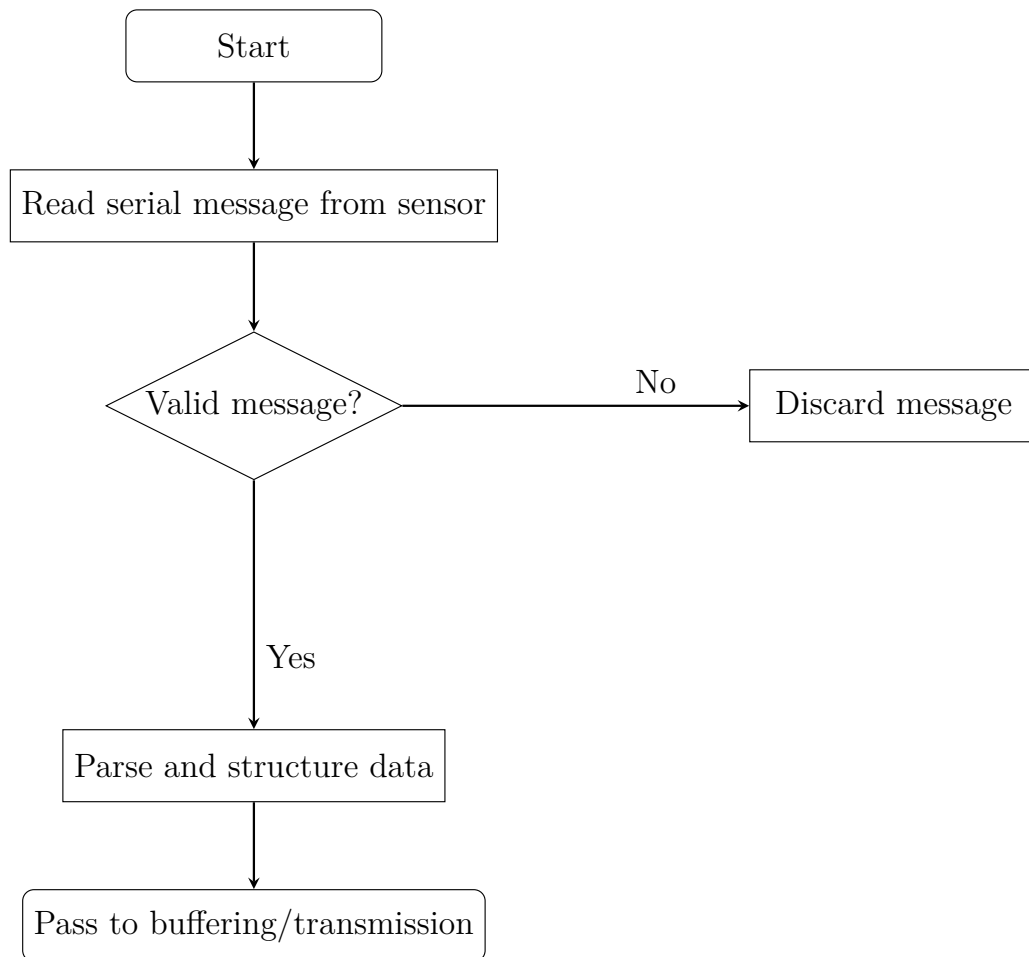


Figure 4.4: Logigram of sensor data acquisition and validation workflow.

4.3.3 Schema Mapping and Output Structuring

After parsing, both sensor data dictionaries are augmented with metadata (such as timestamp and sensor ID) and prepared for transmission. The schema used internally aligns with the backend cloud structure and is converted to JSON before being passed to the GraphQL mutation.

For example, a sample output from the PWD12 parser might be structured as shown in Listing 4.3.

Listing 4.3: Parsed JSON structure from a PWD12 message

```
{  
  "visibility_alarm": "No Alarm",  
  "visibility_1min": 1839,  
  "visibility_10min": 1505,  
  "present_weather_nws": "Light Rain",  
  "present_weather_code_inst": "Rain",  
  "present_weather_code_15min": "Rain",  
  "present_weather_code_1h": "Rain",  
  "precip_intensity": 0.33,  
  "precip_cumulative": 12.16,  
  "snow_sum": 0  
}
```

This structured representation is then either transmitted directly to AWS AppSync or stored in a local SQLite buffer in case of network unavailability, as described in Section 4.4.

The decision process between cloud transmission and local buffering is shown in Figure 4.5.

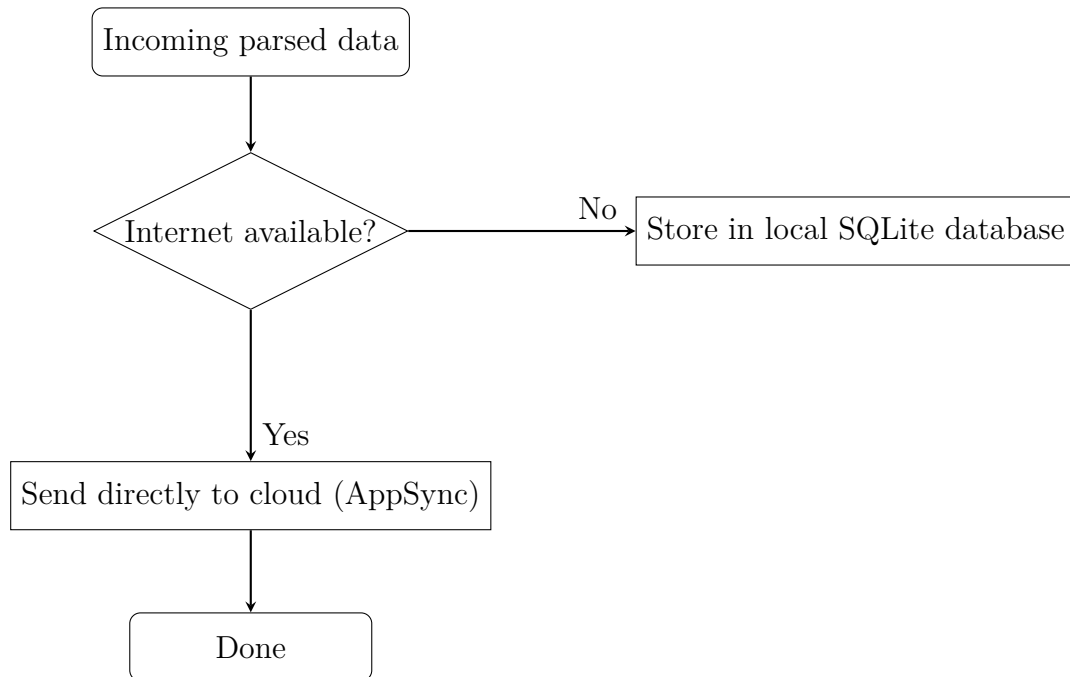


Figure 4.5: Logigram of offline buffering with SQLite.

4.4 Offline Buffering and Local Data Resilience

4.4.1 Problem Statement and Design Requirements

In deployments where cellular connectivity may be unstable or unavailable for long durations, the weather station must guarantee data integrity by buffering locally. The buffering mechanism must:

- Store all sensor data during internet outages without loss
- Automatically retry sending once connectivity is restored
- Prevent duplicated uploads
- Limit storage usage and clean up old entries if necessary

4.4.2 Evaluation of Local Storage Options

Several local buffering mechanisms were considered for storing sensor data during internet outages. Table 4.3 summarizes the trade-offs:

Table 4.3: Local storage option comparison for embedded buffering [10], [11].

Storage Option	Persistence	Ease of Access	Suitability
Volatile Memory Buffer	Lost on reboot or crash	Fast, in-memory operations	Unsuitable: not fault-tolerant
Flat File (e.g. JSON)	Persistent	Requires manual parsing, inefficient updates	Limited: simple but hard to manage
SQLite Database	Persistent	Structured, queryable, concurrent-safe	Recommended: lightweight and robust

SQLite was chosen for its balance between simplicity and power. Unlike flat files, it offers atomic transactions, concurrent-safe access in a multithreaded system, and efficient indexing and cleanup mechanisms. It also requires no external dependencies and fits well into the embedded constraints of the Raspberry Pi system. SQLite was selected because it is serverless, ACID-compliant, robust against power loss when used with journaling, and well-suited to embedded devices [10], [11].

4.4.3 Buffer Architecture and File Management

The buffer is implemented using a single SQLite table named `sensor_data`, created during initialization via `init_db()`. Each record consists of:

- `id`: Auto-incremented primary key
- `timestamp`: Coordinated Universal Time (UTC) epoch time when the data was collected

- `sensor_id`: Unique identifier of the originating sensor
- `payload`: Serialized JSON string of the sensor data

Data is inserted using the `save_to_sqlite()` function after every failed upload attempt or when the system detects that there is no internet connection. Records can be printed, counted, or fetched via helper functions for inspection or synchronization purposes.

4.4.4 Retry Logic and Recovery Workflow

A background synchronization thread runs from startup and enforces FIFO ordering (oldest rows first) so that cloud timelines reflect true sampling order. Connectivity is probed via a lightweight DNS socket check to `8.8.8.8:53`. The workflow is:

1. If connectivity is available, fetch a batch of oldest rows (configurable window, default 100).
2. Attempt upload for each row in order.
3. On success, delete the row with `delete_row_by_id()`.
4. On the first failure, stop the loop and enter backoff before the next attempt.

The retry and recovery procedure is summarized in Figure 4.6.

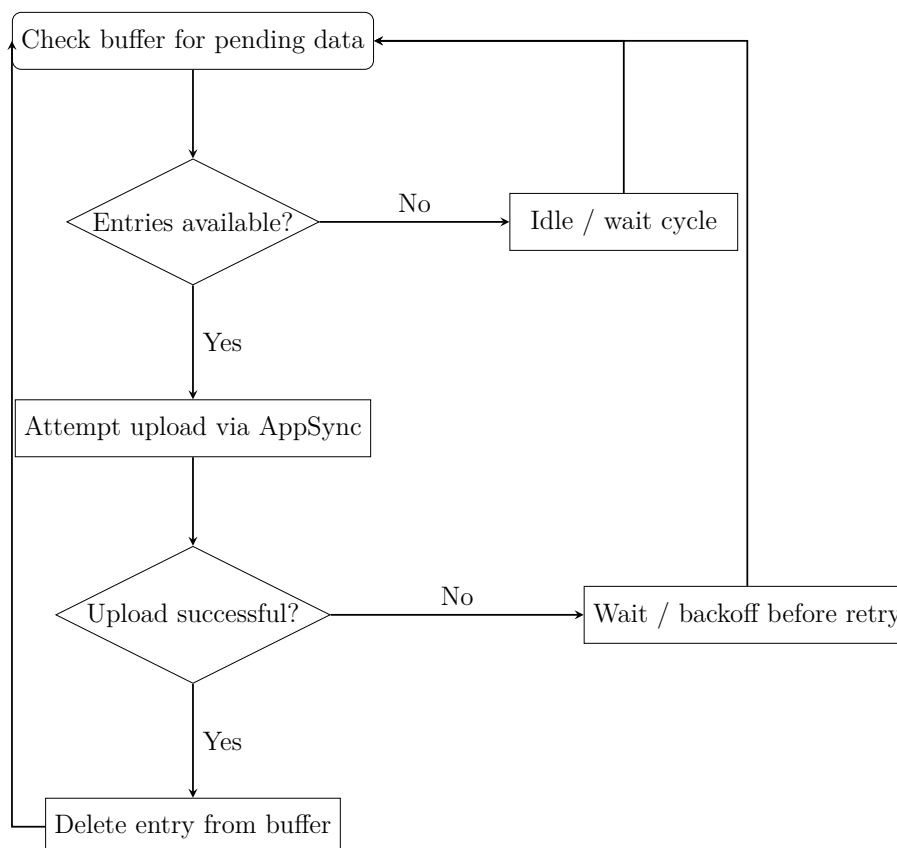


Figure 4.6: Logigram of cloud transmission with retry mechanism.

4.4.5 Edge Case Handling and Failure Modes

Duplicate Prevention Each record includes a precise timestamp, and the system verifies non-null input values with `is_not_null()` to avoid uploading corrupted or duplicate data. The cloud endpoint (AppSync) uses timestamps and TTL fields for consistency and deduplication.

Batching Efficiency Although rows are currently uploaded one by one, the buffer structure supports future batching extensions, e.g., sending up to 25 entries at once using a pipeline resolver.

Max Buffer Size and Cleanup To prevent unlimited growth, a maximum buffer size is enforced via the `MAX_ROWS` constant. Once this threshold is reached, the system deletes the oldest row using the `cleanup_table()` function. This keeps disk usage bounded without requiring external intervention.

4.5 Cloud Communication and Batch Upload

4.5.1 GraphQL Mutation Design

The current deployment uses a single-record mutation that ingests one parsed payload at a time. This keeps the resolver simple and aligns with present throughput requirements.

Listing 4.4: GraphQL schema for single-record weather data ingestion

```
input CreateWeatherDataInput {
  created_at: Float!
  weatherstation_id: String!
  weather_data: AWSJSON!
  expiry_date: Int
}

type WeatherData @aws_api_key
  @aws_cognito_user_pools {
  created_at: Float!
  weatherstation_id: String!
  weather_data: AWSJSON!
  expiry_date: Int
}

type Mutation {
  createWeatherData(input: CreateWeatherDataInput!): WeatherData
    @aws_api_key
    @aws_cognito_user_pools
}
```

```
type Query {  
    getWeatherData(weatherstation_id: String!, created_at: Float!): WeatherData  
        @aws_api_key  
        @aws_cognito_user_pools  
}  
  
type Subscription {  
    onCreateWeatherData(weatherstation_id: String): WeatherData  
        @aws_subscribe(mutations: ["createWeatherData"])  
        @aws_cognito_user_pools  
}
```

4.5.2 AppSync Pipeline Resolver for Batching

Batch submission (e.g., arrays of up to 25 items) can be added later using an AppSync pipeline resolver that validates each item, writes to DynamoDB in a single request, and returns per-item statuses. As the current station traffic does not require batching, this extension is deferred to future work.

4.6 Deployment and Monitoring

The integrated system was designed for stable operation in a remote or semi-autonomous context, such as a field-deployed weather station. Deployment strategy and observability were key to ensuring reliability and maintainability in real-world conditions.

4.6.1 Deployment Strategy for RevPi

The application is deployed on a Revolution Pi Connect+ running Raspberry Pi OS (Bookworm). The software stack, implemented in Python 3.11, is maintained entirely through **remote deployment**, ensuring that updates and configuration changes can be applied without physical access to the device. This is critical in operational environments where the RevPi may be installed in restricted or hard-to-reach locations.

Remote management is enabled via Qbee.io, which provides secure device access, configuration distribution, and automated updates. Standard SSH access is also available as a fallback for manual intervention. Together, these mechanisms ensure that the system can be continuously improved and maintained in the field.

The overall deployment approach follows these principles:

- Source code is version-controlled and hosted in a Git repository.
- Updates are pulled remotely onto the RevPi via Qbee shell or SSH.

- The application (`main.py`) and dependencies are installed in a dedicated working directory (e.g., `/home/revpi/weather_station`).
- Configuration is stored in a local `.env` file, excluded from version control, to allow site-specific customization without code changes.

No containerization layer is used in production, as **systemd services** proved more reliable for startup control, simpler debugging, and lower resource overhead on the embedded device. This strategy provides a lightweight, maintainable, and fully remote deployment workflow, aligning with operational requirements for weather station installations.

4.6.2 Systemd Configuration and Startup Behavior

The weather station runs as a persistent service managed by **systemd**, ensuring automatic startup after reboot and consistent recovery after crashes. An example unit file is shown below:

Listing 4.5: Systemd unit file for PWD12 weather service

```
[Unit]
Description=PWD12 Weather Station Service
After=network.target

[Service]
Type=simple
User=unisphere
WorkingDirectory=/home/unisphere/weather_station_iot
ExecStart=/home/unisphere/weather_station_iot/start_pwd12.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Key systemd behaviors include:

- `Restart=always` with `RestartSec=5` prevents downtime after failures.
- Logs are captured automatically by `journald` and persist across reboots.
- The service runs as a non-privileged user (`revpi`).
- Secrets are stored in a restricted `.env` file, excluded from Git.

In terms of security:

- Only the `unisphere` user has execution permissions.

- Environment variables are stored in `.env`, which is excluded from version control via `.gitignore`.
- The `start_pwd12.sh` script is nonworldreadable.

Operational hardening. The service is sandboxed using `NoNewPrivileges=true` and `PrivateTmp=true`, reducing attack surface. Scripts such as `start_pwd12.sh` are non-world-readable, and environment variables are never written to logs.

4.6.3 Remote Access and Diagnostics

Remote management is handled via Qbee.io, which provides secure access for software updates, service restarts, and log inspection. In practice, developers can:

- Pull updates from Git or deploy patches.
- Restart or monitor `systemd` services.
- Inspect logs with `journalctl -u weather_station.service`.
- Access the device over SSH with restricted user privileges.

These capabilities are essential for updating firmware, applying patches, or performing diagnostics without physical access to the deployment site.

4.6.4 Logging, Metrics, and Alerting

Local logging is handled using standard output via `print()` statements and captured by `journald`. Logs can be queried by timestamp, filtered by service, and persist across reboots.

While no dedicated metrics or alerting system is integrated at this stage, the existing logging includes:

- Sensor readouts with timestamps.
- Cloud publishing outcomes (success or buffer fallback).
- Checksum errors or parsing failures.
- Offline buffering and sync attempts.

Future improvements. While no dedicated metrics or alerting system is yet deployed, options include:

- Lightweight metrics collection via Prometheus node exporter.
- Qbee log forwarding to a centralized dashboard.
- Cloud-based alerting on repeated failures or buffer overflows.

These extensions would enhance observability and long-term operational resilience.

4.7 Documentation and Maintainability

To ensure the long-term sustainability of the integrated weather station system, particular attention was given to documentation, modular code design, and update processes. These elements facilitate onboarding new developers, performing remote maintenance, and enabling future sensor additions.

4.7.1 Developer Documentation and Comments

The source code is fully commented and structured around dedicated modules per sensor (e.g., `pwd12/` and `wxt536/`). Inline documentation follows Python conventions with clear docstrings for functions and classes.

In addition, the project contains:

- A `README.md` with setup instructions and dependencies.
- An annotated `requirements.txt` to specify Python packages.
- Clear environment variable usage via a version-controlled `.env.example` template.
- Diagrams that explain hardware connections and data flow.

This documentation provides the context necessary to understand the architecture, communication protocol, and operational flow without deep code inspection.

4.7.2 Update Strategy and Future Expansion

Updates are applied using a Git-based workflow and deployed remotely via Qbee. The current strategy involves:

- Pulling changes on-device using the Git CLI.
- Updating the systemd service if file paths or execution logic change.
- Validating new features in a test environment before rollout.

The modular structure allows new sensors to be added with minimal changes to the main logic. For example, adding a third sensor would involve:

- Creating a new folder for its reader/parser.
- Launching a dedicated thread in `main.py`.
- Updating the GraphQL payload format accordingly.

This makes the system extensible without requiring a full redesign.

4.7.3 Testing Guidelines and Deployment Checklist

While no formal CI/CD pipeline is implemented, informal testing procedures were followed throughout development. These include:

- Serial port echo tests to validate physical sensor communication.
- Mock data replay from logs to verify parsing logic.
- Cloud sync simulations using disconnected environments.
- Offline buffering threshold stress tests (e.g., >700 entries).

The deployment checklist includes:

1. Ensure all environment variables are defined.
2. Pull latest code and verify permissions (`chmod +x scripts`).
3. Reload and restart the relevant `systemd` service.
4. Monitor logs with `journalctl` for sensor and sync activity.
5. Optionally, trigger a manual cloud sync to verify AppSync connectivity.

These practices provide confidence in stability across both development and production environments.

4.7.4 Files to touch when adding a new sensor.

1. Create `<sensor>/reader.py` and `<sensor>/parser.py`.
2. Register a new thread in `main.py` with sensor ID and port.
3. Extend shared schema mapping (payload builder) if new fields are introduced.
4. Add environment keys to `.env.example` and document in `README.md`.
5. (Optional) Add unit/mock tests for parser edge cases.

4.8 Results

To validate the integration of the Vaisala PWD12 sensor, several outcomes were captured from both the device logs (via Qbee) and the cloud infrastructure (AWS DynamoDB).

4.8.1 Local Logs via Qbee

Figure 4.7 shows a snapshot of the live logs captured from the RevPi device through Qbee’s remote console. These logs provide direct feedback on the end-to-end data pipeline, confirming that sensor messages are correctly acquired, parsed, and published to the cloud.

A typical log entry contains:

- The **timestamp** of the acquisition.
- The **station identifier** (here: IPC2100).
- The **systemd service name** running the process (e.g., `start_pwd12.sh`).
- An **excerpt of the parsed message**, showing visibility, present weather, and precipitation values in JSON-like format.
- The corresponding **AppSync response**, confirming successful ingestion into the cloud backend.

In addition to sensor data logs, the console also shows **buffering events**, for example “No buffered data,” which indicate that the system checked the local SQLite database for unsent entries but found none.

This combination of structured sensor outputs, AppSync acknowledgements, and explicit buffering events demonstrates both the correctness of the parsing logic and the resilience of the upload pipeline under Qbee monitoring.

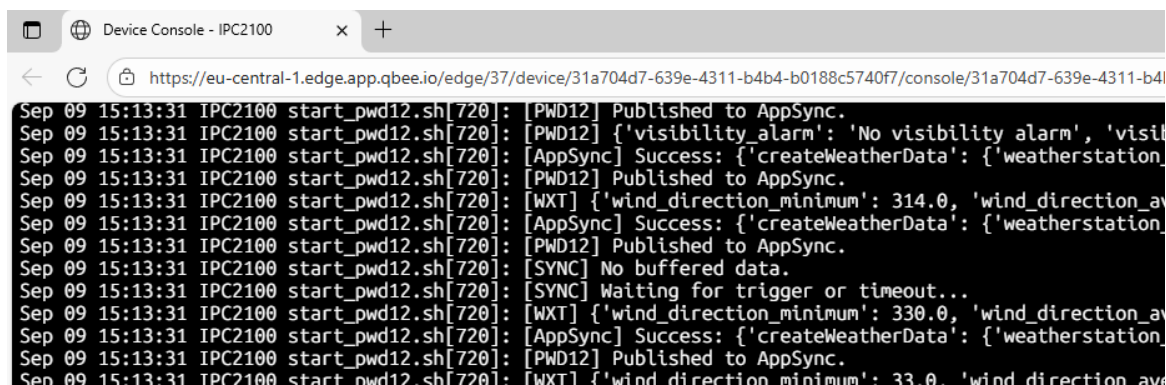


Figure 4.7: Example of live PWD12 sensor logs captured remotely through Qbee, showing parsed data, AppSync responses, and separate buffering events.

4.8.2 Cloud Data on AWS DynamoDB

Once parsed and validated, sensor messages are uploaded to the cloud via AppSync and stored in an AWS DynamoDB table. Figure 4.8 presents a snapshot of the table view, where each row corresponds to a single data record produced by the weather station. The key attributes visible in this view are the `created_at` timestamp (in epoch format) and the `weatherstation_id`, which uniquely identify when and where the measurement was taken. The `weather_data`

field, stored in JSON format, encapsulates the full set of parsed parameters such as visibility averages, present weather codes, precipitation intensity, and snow accumulation. This table therefore represents the chronological archive of all measurements transmitted by the PWD12 sensor, with each record aligned to the structure expected by downstream applications.

☐	weatherstation_id (String)	created_at (Number)	expiry_date (TTL)	weather_data
☐	IPC2100_PWD12	1756198757.384496	1756371557	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil
☐	IPC2100_PWD12	1756198772.374158	1756371572	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil
☐	IPC2100_PWD12	1756198787.365795	1756371587	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil
☐	IPC2100_PWD12	1756198802.356181	1756371602	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil
☐	IPC2100_PWD12	1756198817.350119	1756371617	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil
☐	IPC2100_PWD12	1756198832.360405	1756371632	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil
☐	IPC2100_PWD12	1756198847.345162	1756371647	{ "visibility_10min": { "N": "2000" }, "visibility_alarm": { "S": "No visibil

Figure 4.8: PWD12 sensor data stored in AWS DynamoDB. Each row corresponds to a structured measurement record with timestamp, station identifier, and JSON payload.

For clarity, Figure 4.9 zooms into one specific record. This detailed view highlights how the parsed payload is stored in DynamoDB: the JSON string contains key-value pairs that mirror the structure defined in Section 4.3. For example, the fields `visibility_1min` and `visibility_10min` provide rolling averages of measured visibility, while `present_weather_nws` and `present_weather_code_inst` capture instantaneous and coded weather conditions. Precipitation values, such as `precip_intensity` and `precip_cumulative`, are also preserved. In addition, metadata such as the station identifier ensures that data from multiple deployed stations can be distinguished on the backend.

☐ Attribute name	Value	Type
weatherstation_id - Partition key	IPC2100_PWD12	String
created_at - Sort key	1756198607.424043	Number
expiry_date	1756371407	Number
☒ weather_data	Insert a field ▼	Map
precip_cumulative	0	Number
precip_intensity	0	Number
present_weather_code_15n	Clear	String
present_weather_code_1h	Clear	String
present_weather_code_inst	Clear	String
present_weather_nws	No Precipitation	String
snow_sum	0	Number
visibility_10min	2000	Number
visibility_1min	2000	Number
visibility_alarm	No visibility alarm	String

Figure 4.9: Detailed view of a single PWD12 sensor record in DynamoDB.

Together, these two figures demonstrate the complete data persistence workflow: sensor messages generated by the PWD12 are parsed locally on the RevPi, buffered when connectivity is unavailable, and then ingested into DynamoDB in a structured format. The result is a reliable, queryable time series of weather data that can be consumed by higher-level services for monitoring, analytics, or flight planning. This confirms both the correctness of the parsing logic and the robustness of the cloud synchronization pipeline.

4.8.3 Discussion of Results

The screenshots illustrate that:

- Sensor data is correctly read and parsed on the RevPi.
- Offline buffering and retry logic function as intended when connectivity is intermittent.
- Cloud synchronization to AWS AppSync and DynamoDB works reliably, with proper schema mapping and timestamp ordering.

These results confirm that the PWD12 integration provides operational visibility measurements in real-time, both locally and in the cloud, fulfilling the system design objectives.

4.8.4 Field Validation and Bug Fixing

To validate the integration in real-world conditions, the system was temporarily deployed on a rooftop installation in Konstanz. This short-term test allowed direct exposure of the sensors to outdoor weather, providing more realistic operating conditions compared to bench testing.

During this deployment, several bugs were discovered, primarily related to parsing edge cases and intermittent communication errors. These were fixed iteratively, resulting in a more robust software pipeline capable of handling sensor noise and temporary data gaps.

No pictures were recorded during this short-term rooftop test, but operational logs and observations confirmed the system's functionality under real conditions. Representative log excerpts are shown in Figure 4.10, highlighting successful message acquisition, parsing, and cloud transmission.

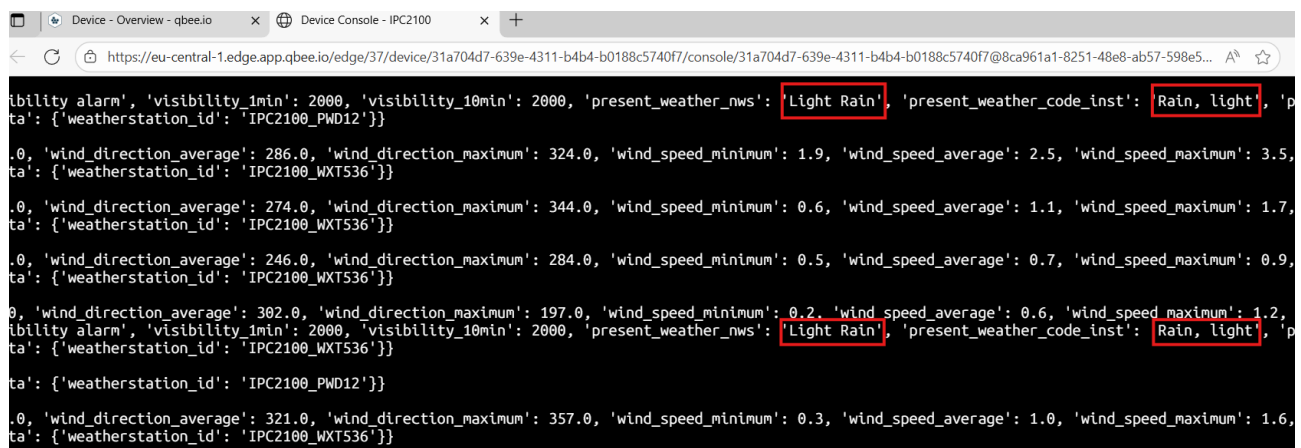


Figure 4.10: Excerpts from live logs during the rooftop field test.

Conclusion

This chapter presented the complete integration of the Vaisala PWD12 sensor into the existing weather station system. Hardware adaptations enabled stable RS-485 communication, while modular Python components were developed for reading, parsing, and structuring sensor data. Robustness was ensured through local SQLite buffering with retry logic, and cloud synchronization was achieved via AppSync GraphQL mutations. Deployment under systemd provides long-term stability, security, and autonomous operation.

General Conclusion

This thesis presented the design and implementation of an **IoT-based weather monitoring solution** that extends the capabilities of an existing weather station at Unisphere GmbH. The overall objective was to integrate visibility and present weather measurements while increasing the resilience of the system against temporary network disruptions.

The work began with a study of the deployed infrastructure, which originally consisted of a single multisensor unit connected to a Revolution Pi industrial controller and synchronized with the cloud through AWS AppSync. While effective for parameters such as wind, temperature, humidity, and precipitation, this setup lacked both visibility monitoring and offline buffering, two features that are essential for reliable operation in the context of aviation and urban air mobility.

To address these shortcomings, a new visibility sensor was examined in detail. Its measurement principle, installation constraints, and communication formats were analyzed, followed by bench tests that confirmed stable RS-485 operation at 9600 bps. A parsing logic was implemented to convert raw ASCII messages into structured JSON records aligned with the cloud schema. This ensured that data generated locally could be seamlessly integrated with Unisphere's existing cloud services.

Integration into the operational weather station required hardware and software adaptations. On the hardware side, the sensor was connected via a dedicated serial interface, with only power and RS-485 lines wired through an M12 connector. On the software side, modular Python components were developed to separate sensor reading, message parsing, and data serialization. These processes were executed under **systemd** to guarantee autonomous operation, automatic restart on failure, and secure long-term deployment.

A critical contribution of this work was the addition of a local SQLite buffer. This mechanism stores data whenever connectivity is lost, retries synchronization once the network is restored, and guarantees the temporal order of measurements without duplication. Validation confirmed that several hundred entries could be buffered and recovered successfully, demonstrating a significant improvement in reliability compared to the previous version of the system.

The complete workflow was validated through live Qbee logs and DynamoDB records, which confirmed successful acquisition, parsing, buffering, and synchronization of sensor data. Rooftop tests provided further evidence that the system functions correctly under real environmental conditions, with continuous acquisition and cloud delivery confirmed over several days of operation.

Beyond these results, several perspectives open the way for future developments. At the cloud level, the GraphQL schema could be extended to support batch ingestion, improving efficiency for high-frequency deployments. A formal monitoring and alerting layer would strengthen operational reliability by enabling proactive maintenance. At the hardware level, additional sensors such as ceilometers could be integrated to enrich the range of meteorological parameters. Finally, scaling the solution across multiple stations would reinforce Unisphere’s U-Space weather services and provide reliable, localized data for flight planning and traffic management.

In conclusion, this work delivered a resilient and extensible IoT weather monitoring system that enhances measurement coverage and ensures robust data availability. The results demonstrate the feasibility of integrating advanced sensors with industrial controllers, local buffering, and cloud synchronization in a modular framework. Such solutions are key enablers for safe and efficient operations in aviation and emerging urban air mobility environments.

Bibliography

- [1] Unisphere GmbH, *Linkedin profile*, <https://www.linkedin.com/company/unisphere-gmbh>, Accessed: 2025-07-10.
- [2] Unisphere GmbH, *Handelsregister (german commercial register) entry*, Official company registration data. Accessed: 2025-07-10.
- [3] Unisphere GmbH, *Company website*, <https://unisphere.ai>, Accessed: 2025-07-10.
- [4] *Aviation and drone industry news articles*, e.g., UAV Industry News. Accessed: 2025-07-10.
- [5] Solar Impulse Foundation, *Solar impulse - a flying laboratory for clean technologies*, <https://solarimpulse.com/>, Accessed: 2025-07-10.
- [6] Skyports Infrastructure, *Skyports and rta break ground on first vertiport in dubai*, <https://skyports.net/skyports-and-rta-break-ground-on-first-vertiport-in-dubai/>, Accessed: 2025-07-10.
- [7] AINonline, *Skyports' first uae vertiport earns technical design approval*, <https://www.ainonline.com/aviation-news/futureflight/2025-01-15/skyports-first-uae-vertiport-earns-technical-design-approval>, Published: 2025-01-15. Accessed: 2025-07-10.
- [8] *Present weather detector pwd22/52 user's guide*, Vaisala, Finland; retrieved from datasheet, Vaisala Oyj, 2015. [Online]. Available: https://www.qiometer.com/upfile/datasheet/PWD22_52%20Users%20Guide%20M210543EN-F.pdf
- [9] *Present weather detector pwd12 user's guide*, Accessed September 13, 2025, Vaisala Oyj, 2025. [Online]. Available: <https://docs.vaisala.com/v/u/B210385EN-G/en-US>
- [10] D. R. Hipp and S. Consortium, *Appropriate uses for sqlite*, <https://www.sqlite.org/whentouse.html>, Accessed 2025-08-19.
- [11] D. R. Hipp and S. Consortium, *Atomic commit and durability in sqlite*, <https://www.sqlite.org/atomiccommit.html>, Accessed 2025-08-19.



REPUBLIC OF TUNISIA

Ministry of Higher Education and Scientific Research

University of Carthage

National Institute of Applied Sciences and Technology



Development of an IOT-based Weather Sensor System for Drone Operations

Abstract

This graduation project presents the integration of the Vaisala PWD12 visibility sensor into an existing weather station originally based on the WXT536 sensor. Deployed on a Revolution Pi controller, the system delivers real-time meteorological data for Unisphere GmbH's NOVA platform. The hardware ensures robust RS-485 connectivity, while the modular Python software architecture provides dedicated readers and parsers managed by systemd. Local buffering with SQLite guarantees data persistence and reliable synchronization with the AWS AppSync API. The final system is resilient, extensible, and capable of supporting additional sensors with minimal effort.

Keywords: visibility sensor, Vaisala PWD12, weather station, RS-485, Python, offline buffering, AWS AppSync

Développement d'un système de capteur météo basé sur l'IoT pour les opérations de drones

Résumé

Ce projet de fin d'études décrit l'intégration du capteur de visibilité Vaisala PWD12 dans une station météo existante initialement basée sur le capteur WXT536. Déployé sur un contrôleur Revolution Pi, le système fournit des données météorologiques fiables et en temps réel pour la plateforme NOVA d'Unisphere GmbH. Le matériel assure une connectivité RS-485 robuste, tandis que l'architecture logicielle modulaire en Python propose des lecteurs et parseurs dédiés, gérés par systemd. Le stockage local avec SQLite garantit la persistance des données et leur synchronisation fiable avec l'API AWS AppSync. Le système final est résilient, extensible et peut intégrer facilement d'autres capteurs.

Mots-clés : capteur de visibilité, Vaisala PWD12, station météo, RS-485, Python, stockage local, AWS AppSync

تطوير نظام استشعار الطقس المبني على إنترنت الأشياء لعمليات الطائرات بدون طيار

ملخص

يتناول مشروع التخرج هذا دمج مستشعر الرؤية Vaisala PWD12 في محطة طقس قائمة تعتمد أساساً على مستشعر WXT536. يتم تشغيل النظام على وحدة تحكم Revolution Pi لتوفير بيانات أرصاد جوية دقيقة وفورية لمنصة NOVA التابعة لشركة Unisphere. يتضمن التصميم المادي اتصالاً متيناً عبر RS-485، بينما يعتمد البرنامج على بنية معيارية بلغة Python مع وحدات قراءة وتحليل مخصصة تُدار بواسطة systemd. يتضمن التخزين المحلي باستخدام SQLite حفظ البيانات ومزامنتها بشكل موثوق مع واجهة برمجة التطبيقات AWS AppSync. النظام الناتج مرن وقابل للتوسيع ويمكنه دعم أجهزة استشعار إضافية بسهولة.

الكلمات المفتاحية: مستشعر الرؤية، Vaisala، PWD12، محطة طقس، RS-485، Python، التخزين المحلي، AWS AppSync